Journal of Software: Evolution and Process

WILEY

Software: Evolution and Process

REVIEW ARTICLE OPEN ACCESS

Integrating Security Controls in DevSecOps: Challenges, Solutions, and Future Research Directions

Maysa Sinan | Mojtaba Shahin | Iqbal Gondal

School of Computing Technologies, RMIT University, Melbourne, Victoria, Australia

Correspondence: Maysa Sinan (Maysa.sinan@gmail.com)

Received: 19 July 2024 | **Revised:** 23 January 2025 | **Accepted:** 5 May 2025

Funding: Open access publishing facilitated by RMIT University, as part of the Wiley—RMIT University agreement via the Council of Australian University Librarians.

Keywords: cybersecurity | DevOps | DevSecOps | security challenges | security controls

ABSTRACT

Cybersecurity has become a top priority for most organizations to protect their applications. The rapid increase in cyberattacks has necessitated a comprehensive repositioning of how security should be implemented within the software development life-cycle (SDLC). Development, Security, Operations (DevSecOps) is one of the trendy security methodologies and fastest growing development methods promoting shared responsibility for security and automating security practices at every step of the SDLC. DevSecOps is a cultural shift that integrates security controls into DevOps pipelines aiming to upscale overall security. Therefore, many organizations started to incorporate security controls within the deployment of DevSecOps through conducting continuous practices, for example, automated security testing, infrastructure as code (IaC), compliance as code, and continuous monitoring. This study aims to organize the knowledge and shed light on challenges concerning security controls during the adoption of DevSecOps, along with associated solutions and remediation workarounds reported in the literature. Further, the study aims to provide clear insights into the areas that require further investigation and research in the future. A systematic literature review (SLR) of 45 primary studies was carried out to extract data, and subsequently, the extracted data was analyzed using the thematic analysis method. This paper identifies 19 challenges related to security controls that could be experienced by security practitioners while implementing a DevSecOps model, along with 18 solutions and remediation actions suggested in literature to address and overcome some of the enlisted challenges. In addition, some gap areas are identified as opportunities for future research in this domain with the aim of improving the integration of security controls in a DevSecOps environment. Based on findings, this paper points out the importance of automation in software engineering practices, for example, continuous automation, continuous delivery, and continuous feedback, to embed security controls at the early stages of the development process.

1 | Introduction

Cybersecurity has become necessary within the SDLC in response to increasing threats [1]. In traditional delivery methods, IT Service Management (ITSM) supports software applications by developing end-to-end delivery processes, keeping an eye on compliance and adherence to organizational policies [2, 3]. In such models (e.g., waterfall), security controls are an afterthought, limited in scope, and often tackled and tweaked by security teams after quality assurance (QA) processes are completed [4]. On the other hand, Software Development as a Service (SDaaS) has become crucial for many organizations due to its flexibility, cost-effectiveness, and the efficiency it brings to the software development process [5]. Moreover, SDaaS allows companies to scale up or down based on project needs, avoiding the high costs associated with recruiting or

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2025 The Author(s). Journal of Software: Evolution and Process published by John Wiley & Sons Ltd.

downsizing staff. It also enables software engineers to stay up to date with the latest technologies, ensuring that their projects and applications are developed with best practices and cutting-edge tools. In the DevSecOps methodology, security analysts progressively integrate and streamline security controls at every phase of the SDLC (from design to build) with minimal disruption [6]. DevSecOps aims to automate continuous security practices in DevOps layers and empower the mantra "software, safer, sooner" during the delivery process without slowing down deliveries [7]. The hypothesis of enhancing security seamlessly within DevOps activities as agile as possible has contributed to the main concept of DevSecOps [8]. Given this, developers are encouraged to take responsibility and participate in security analysis and discussions related to application infrastructures. They are expected to flag insecure choices, present effective findings, and escalate/share contingency workarounds with DevSecOps engineers for future improvements [9, 10].

DevSecOps and agile are complementary methodologies that provide the opportunity to left-shift security and build software in tandem with continuous security practices. Such methods require high levels of effective team collaboration to maintain security compliance and accelerated agile work cycles until applications are implemented at customer sites [11]. DevSecOps also highlights the need for knowledge sharing to achieve a superior security culture. Researchers conducted studies on Open Web Application Security Projects (OWASP) to develop reliable approaches such as the survival guide pyramid to overcome the limitations of obsolete security practices that might decrease the benefits of DevSecOps [12], as in Table 1.

Security controls in DevSecOps interfaces are indispensable to ensure a robust security posture throughout the software development lifecycle (SDLC). They are essential to protect sensitive data while meeting regulatory and security requirements [13].

TABLE 1 | The survival guide pyramid of James Wickett [12].

Underpin	Description
Defensible	Solid security controls and configuration settings are put in place with consistency
Operational	Transactional data are managed collaboratively with the focus on security awareness and effective interaction between development and operation teams
Situational awareness	System updates are monitored closely to determine potential cyberattacks and adversary effects
Contradict measurements	Efficient responses are prompted while preventive plans and communication strategies are held

Security controls integrated early in the development process help detect and address security weaknesses and vulnerabilities at the initial stages, reducing the cost and effort of fixing them later. On the other hand, they are important to facilitate uninterrupted delivery and execution of continuous practices without compromising overall security [14]. Integrating security controls into CI/CD pipelines ensures that security-as-code audits are integral part of development process [15]. In contrast, immediate feedback on security issues allows software engineers and developers to address issues quickly while maintaining a stable pace of development.

1.1 | Aim and Contribution

Shifting from conventional methods to DevSecOps and addressing security issues at early stages are declared challenging [16]. It is indispensable for security practitioners to explore existing issues concerning security controls that may arise during the implementation of DevSecOps and hinder the development process. This systematic literature review (SLR) is designed to provide value not only to software engineers but also to professionals from related disciplines and roles, including infrastructure engineers, operations engineers, security engineers, and site reliability engineers (SREs). The recommendations related to security controls can address the challenges and opportunities that may arise during the integration of security controls within DevSecOps. The paper classifies the challenges related to security controls and thematically analyzes the suggested solutions proposed in the literature to address the reported challenges/issues. Security practitioners can use it as a base to conduct further research on implementing security controls, as it explains the research gaps related to building security controls in continuous operation of DevSecOps. From SLR studies, valuable lessons can be learned to drive innovation and improvements in DevSecOps processes [17].

2 | Background and Related Work

This section provides the background and summarizes the existing review papers concerning security controls in DevSecOps.

2.1 | Security Controls

Security controls, or security measures, are specific mechanisms/safeguards used to protect organizational assets and systems from cyberattacks. They are fundamental components implemented to mitigate vulnerabilities in software applications and reduce the impact of security incidents in tandem for better data protection, risk assessments, and security compliance with the guidelines outlined by National Institute of Standards and Technology (NIST) and International Organization for Standardization 27001 (ISO 27001) frameworks [18, 19]. Security controls are usually customized based upon organizational regulations, risk profiles, and compliance strategies [20, 21], which can be categorized as follows:

1. *Administrative controls* that manage security practices including the support for NIST and ISO 27001 standards toward managing the quality of the information in Security Management Systems enterprise (SMSE) [22]:

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

a. Security policies and procedures (e.g., formal documents and organizational guidelines).
b. Security Education Training Awareness (SETA) to educate employees about security best practices and prevalent threats.
c. Access control policies (ACP) for provisioning and deprovisioning user access requests to systems and data.
d. Incident response management (IRM) to develop effective response plans and promptly defending against attacks.
e. Risk assessments to identify security risks and run periodic evaluations.

2. *Technical controls* that are implemented through software automation to prevent developers from risky behaviors [23]:
   a. Authentication and authorization mechanisms to verify user identities and check access requests.
   b. Intrusion detection/prevention systems (IDS/IPS) to monitor, detect, and protect networks from unauthorized access or malicious activities.
   c. Encryption methods to convert data into cipher texts and make information accessible only to authorized users with special decryption keys.
   d. Antivirus and antimalware functions to detect and remove malicious software.
   e. Patch management to ensure that systems are up to date with security.
   f. Security information and event management (SIEM) to conduct real-time security analysis for incidents and events.

3. *Physical controls* that protect materialistic assets [24]:
   a. Access control systems and secure facilitates to restrict entry/access to data centers (e.g., keycards and biometrics).
   b. Install surveillance and monitoring security cameras to detect unauthorized physical access.
   c. Environmental control systems to adjust/regulate server temperature, humidity, and fire protection.

4. *Network security controls* to protect networks [25]:
   a. Firewall devices to filter/control incoming and outgoing traffic and protect against malicious threats and web-based attacks [26].
   b. Virtual private networks (VPNs) to secure remote access and data transmission traffic.
   c. Apply network segmentation to divide the network into segments and isolate and protect sensitive systems.

5. *Application security controls* to protect onboarded software applications [27]:
   a. Carry out validation procedures to ensure user inputs are thoroughly checked for vulnerabilities.
   b. Implement manual and automatic code reviews to identify security bugs.

6. *Data security controls* like data loss prevention (DLP) and data encryption algorithms to protect data at rest and in-transit modes from unauthorized data sharing and avoid data breaches [28].

7. *Cloud security controls* like identity and access management (IAM), setting up configuration rules and installing encryption methods to improve user authentication, authorization, and access control within cloud-based systems [29].

8. *Endpoint security controls* like endpoint detection and response (EDR) and antivirus software mechanisms installed to monitor data and transactions against malware infections, and fasten/secure individual devices (computers and mobile devices) [30, 31].

9. *Security control guidelines* like complying with NIST or CIS CSF and/or ISO27001 [32].

10. *Third-party risk management* to manage/support security controls with third parties and partners with regular access to systems [33].

2.2 | Standards and Best Practices for Implementing Security Controls

Security controls are implemented as part of an organization's information security management system (ISMS) to ensure compliance with legal requirements, support business continuity, and reduce risks to security triad (confidentiality, integrity, and availability [CIA]). Due to their significant role in helping organizations to manage and to protect information assets, *ISO 27001:2022* framework identifies 93 security controls and classifies them into 4 control groups [34]. *Organizational controls*: 37 controls covering areas like information security policies, governance, risk management, and business continuity [35]. *People controls*: 8 controls relating to training, roles, responsibilities, and user access management [36]. *Physical controls*: 14 controls addressing physical security of assets, secure areas, and environmental threats. *Technological controls*: 34 controls concerning technical mechanisms like cryptography, access controls, and system monitoring [37].

Development of suitable security environment in the organizations is essential to ensuring seamless integration of security controls [38]. Some security vulnerabilities result from unplanned activities and random workarounds, resulting in complex codes and inevitable bugs. Therefore, developers are advised to continuously conduct code refactoring and follow security procedures to protect applications from being vague or goofy [39]. Security analysts use techniques like "code refactoring" to apply changes to the software without changing the visible behavior. According to Fowler, when software engineers apply code refactoring relentlessly, they stimulate better comprehensive design ideas, resulting in enhanced resilience and software recovery [40]. Afterward, white-box and black-box testing is done to handle any unclear patterns resulting from the software refactoring process [41]. Implementing security controls requires a holistic approach that includes a set of secure development practices to stay compliant with security standards and industry frameworks, ensuring that organizations meet both legal and operational security requirements [42]. Below are some of the best practices categorized into different aspects of security control implementation:

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

1. *Risk analysis and assessment*. A proper risk assessment helps to prioritize security controls. Hence, it is crucial to understand the risks and threats before implementing the security controls [43].
   a. Regular risk assessments: Identify risks and vulnerabilities to understand which assets need protection and evaluate the severity of potential threats.
   b. Identify data and systems: Classify data based on sensitivity (e.g., public, private, and confidential), allowing appropriate security controls to be applied based on the risk.
   c. Threat modeling: Simulate potential threats and malicious attacks to identify security gaps within the organization's security posture.
2. *Apply authentication and authorization techniques*. The following key components are implemented to manage user access controls to systems and software applications [44]:
   a. Multifactor authentication (MFA) (e.g., passwords, tokens, and biometric verification), to reduce unauthorized access and the risk of compromised credentials.
   b. Role-based access control (RBAC), to limit user permissions based on their job functions and responsibilities.
   c. Strong password policies like using complex, unique passwords and enforcing regular password changes.
3. *Data encryption methods*. Encrypting data are essential to protect sensitive data stored on servers, databases, or cloud environments in rest mode, transit, and runtime. Further, software practitioners must ensure that the data transmitted over networks (e.g., emails and web traffic) are always encrypted using secure protocols such as TLS/SSL[1] [45, 46].
4. *Security monitoring and IRM*. Continuous monitoring and the ability to respond quickly to incidents and cyberattacks are crucial for mitigating the damage.
   a. Use SIEM systems to monitor suspicious activities, generate alerts, and provide real-time threat detection [47].
   b. IDS/IPS to monitor network traffic for unusual patterns and automatically block suspicious activities.
   c. Develop a solid incident response (IR) plan that outlines how to detect, respond to, and recover from cyberattacks.
   d. Enable regular log and audit to maintain audit trails and utilize them to investigate potential breaches and identify new threats.
5. *Compliance and regulatory adherence*. Security practitioners should conduct security audits frequently to assess the effectiveness of implemented security controls and ensure compliance with legal, industrial requirements and security standards [48], like GDPR,[2] ISO 27001, or NIST.
6. *Security training and awareness*. Organizations should always ensure that their users and team members are well informed about security controls and best practices because human errors are hard to control, especially in the security area [49].
   a. Conduct regular security training for all employees and team members, focusing on common cyberattacks like phishing, insider threats, and social engineering.
   b. Test employees' abilities to enhance security awareness and detect phishing attempts through phishing simulations.
   c. Foster a strong security culture where employees feel responsible and empowered to report suspicious activities.

To achieve a comprehensive security control implementation, security practitioners often utilize a combination of the following standards depending on organizational requirements, business industry, and geographic region:

1. *ISO/IEC 27001*[3] *ISMS*: To establish security requirements, identify access controls, and determine supplier relationships, in tandem with focusing on CIA.
2. *NIST Cybersecurity Framework (CSF)*: To manage IR and security risks, comprising of the five core functions (identify, protect, detect, respond, and recover).
3. *Center for Internet Security (CIS) Controls*: To provide easy-to-implement guidelines and prioritize the best cyber practices for small- and medium-sized businesses (SMBs), in critical areas like inventory management, data protection, and IR.
4. *GDPR*: To primarily mandate security controls for protecting personal data.
5. *Control Objectives for Information and Related Technologies (COBIT)*: To offer a governance framework to manage and align IT goals with business objectives.

2.3 | DevOps and IEEE 2675-2021 Standards

DevOps is a methodology comprising of a set of practices that combines software development (Dev) and IT operations (Ops) to streamline all phases within the SDLC. It promotes a culture of collaboration between development and operations teams to enable faster, more reliable, and automated delivery of projects and software applications development [50]. In addition, DevOps is used to increase an organization's agility and abilities to deliver high-quality software applications efficiently while meeting user expectations. It is basically used to break down the traditional practices between these teams, emphasizing continuous integration (CI), continuous delivery (CD), and automated essential and repetitive tasks like testing and rapid feedback [51]. As a consequence, DevSec ensures that code changes are automatically tested and ready for production release, thus allowing faster and more reliable deployment. Further, DevOps ensures that applications are continuously monitored, and the teams are consistently able to detect issues in real-time and implement solutions before they impact users [52].

The IEEE 2675-2021 Standard is designed to provide directions and useful guidance on building and maintaining secure and reliable software systems using DevOps practices. It provides a comprehensive framework that focuses on integrating security, reliability, and scalability into each phase within the SDLC, ensuring that applications and software systems remain resilient even as they evolve. In addition, this model outlines the best practices and principles for developing and maintaining IT

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

systems within the context of DevOps [53], offering the following key benefits [54]:

1. *System security*. Integrating and embedding security controls across the entire SDLC.
2. *System reliability*. Ensuring that applications and systems are resilient even during disruptions and failures.
3. *System scalability*. Ensuring that software systems can handle the market expansion and growth in demand.
4. *QA*. Implementing continuous practices like testing, monitoring, and verification procedures to maintain high-quality systems.

By leveraging DevOps practices and methodologies in conjunction with the IEEE 2675-2021 Standard, software teams can build systems that are secure, resilient, and scalable in response to IT changes and adversity.

1. *Access control*. Use RBAC to ensure that only authorized personnel have access and can modify systems [55].
2. *CI*. Implement CI pipelines ensures that code changes are automatically maintained and tested [56].
3. *CD*. Deploy applications and embed rollback mechanisms automatically in various environments (e.g., staging and production) to ensure a smooth deployment process using infrastructure-as-code (IaC) tools [57].
4. *Automated security testing (AST) and static application security testing (SAST)*. Use tools like Checkmarx[4] to ensure consistent builds with validation, version control, and artifact management [56].
5. *Security hardening*. Apply security hardening practices (e.g., vulnerability scanning, minimal base images, and image signing) to ensure all components and dependencies are verified against vulnerabilities and managed securely using package managers.
6. *Monitoring and alerting*. Set up real-time monitoring for both security events and performance metrics using SIEM tools [58].

2.4 | DevSecOps

DevSecOps is an extension of DevOps that integrates security practices into DevOps [59]. While DevOps emphasizes collaboration between development (Dev) and operations (Ops) teams to improve the efficiency, speed, and quality of software delivery, DevSecOps incorporates security controls as a core element of the SDLC, rather than dealing with security as a separate process at the end [60].

DevSecOps aims to rectify software and resolve issues as soon as they emerge before they become complex or expensive to fix in production sites [61]. In the DevSecOps context, security controls are used consistently [62], and security practices (e.g., threat modeling and risk analysis) are automated by engaging security experts in the development process [63]. This makes security a shared responsibility between teams (development, security, and operations) rather than segregating security into a separate silo [64]. Stimulating effective collaboration between teams on multiple levels provides an opportunity to work on different issues, for example, service-level agreements with customers, capacity management, and resource availability for applications [12]. DevSecOps does not mean replacing security with development and/or operations or expecting development and/or operations users to become security specialists. Likewise, there is no expectation for security experts to become developers and/or operational users [65]. However, DevSecOps is still a necessary approach used to upgrade overall security within an organization based on four fundamentals; (i) *culture* acquired from working collaboratively with security teams to build/manage security controls [7], (ii) *automation* for security practices is essential to follow up with user feedback and keep up both pace and agility achieved by DevOps in former instances [66], (iii) *measurements* are important to detect real-world threats and track vulnerabilities throughout the SDLC allowing teams to revalidate and assess the reliability of applications [67], and (iv) *sharing* knowledge and DevOps frameworks to build security controls based upon clear contents, perceptible user feedback, and lessons learned from past experiences [68, 69].

DevSecOps has resulted from the many attempts to mitigate/overcome security problems and upgrade security to a higher and centralized position in the SDLC by executing the following processes [70]:

1. *Security shift left*: DevSecOps shifts security "left," meaning security controls are integrated from the beginning of the SDLC. Security tools, checks, and practices are embedded early in the development (coding) phase to identify vulnerabilities and risks sooner, ensuring the code is securely written/tested and resolution time is reduced [71].
2. *Enable zero-trust frameworks*: This approach is based on using security orchestration to promote authentication and authorization, enforcing automated responses and dynamic endorsement for access requests.
3. *Automate testing for access requests*: Automated testing for access requests within every sprint involves integrating access control verification into the CI/CD pipelines to ensure that security controls, such as RBAC and permissions, are thoroughly validated during each development cycle. This is essential for agile environments and rapid development cycles that may introduce several security vulnerabilities.
4. *Enable security scanning for vulnerabilities*: This approach allows developers and software engineers to work without the need to rewrite the code if sudden issues arise. This exercise involves integrating processes to automatically identify potential security weaknesses before, during, and after the software releases. It includes static code analysis, dynamic application testing, dependency scanning, and infrastructure vulnerability assessments [72].
5. *Deploy code changes in small patterns and deliver in iterations*: Small, frequent deployments allow teams to quickly identify bugs, security vulnerabilities, or performance issues in new code changes. Consequently, end-users and stakeholders provide real-world feedback when new features or updates are released iteratively, enabling

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

developers to respond and adjust errors quickly [73]. This technique is crucial for agile methodologies and referred to as "incremental development" or "continuous delivery," bringing several benefits to the development teams and overall security.

6. *Empower developers to prioritize code*: This significantly enhances both individual productivity and team efficiency by giving developers more autonomy and decision-making power regarding which technical tasks to prioritize and focus on. They are more likely to address technical issues and refactor inefficient code because they see the long-term impact, taking more ownership of the quality and sustainability of code. As a result, they would feel a greater sense of responsibility for the success or failure of the project, leading to higher commitment and accountability [74].
7. *Seek security compliance*: Carrying out regular security reports and automated validations is a proactive approach to maintaining security compliance, improving the organizational security posture, and building trust with stakeholders. These reports support better decision-making, increase accountability, and help ensure that the organizations remain compliant with evolving security standards and regulatory requirements [75].

Building DevSecOps practices in line with ISO/IEC 12207:2017 security standards creates a strong synergy between agility and standardization. This results in a more robust software development process, combining the flexibility and speed of DevOps with the comprehensive processes of ISO/12207 [76]. ISO/IEC 12207:2017 ensures that organizations meet international security standards and regulatory requirements, especially where strict compliance is necessary. Moreover, DevSecOps practices help automate and streamline compliance processes while maintaining reliable alignment with the ISO standards, ensuring both reliability and scalability for high-quality systems and mission-critical projects.

2.5 | Security Controls with CI/CD Pipelines

CI is a composite process of aggregating code into a complete product. At the same time, CD is the process where code is checked for errors and reviewed iteratively to action changes/updates before releasing to customer site [77]. Developers may find CI/CD processes tedious as they may prolong the project timeline, risks and costs [78]. SAST and dynamic application security testing (DAST) can be used into CI/CD pipelines to further enhance security-by-design controls [78, 79]. Also, IaC paradigm can be used to incorporate security controls within the CI/CD channels for automated configuration [80]. Embedding security controls within CI/CD pipelines requires professional teams to automate security practices throughout different phases without compromising project timelines [78], to conduct essential activities like IRM, vulnerability scanning, and troubleshooting to detect/analyze bugs and prioritize, based on their impacts [81]. Integrating security controls within CI/CD pipelines can result in many challenges concerning security compliance and adherence to organizational regulations due to a lack of effective communication between developers and operators [82].

2.6 | Existing Reviews on DevSecOps

This SLR's contribution and findings of existing reviews are described in Table 2. Former research discussed challenges in DevSecOps concerning tools, people, or processes. There is no discussion/debate on issues concerning security controls in DevSecOps. For example, Rajapakse et al. discussed the challenges of DevSecOps adoption and introduced them in themes (people, practices, tools, and infrastructure) along with the solutions suggested for enlisted challenges [72].

Hvardand and Ricardo described the benefits of DevSecOps practices and explained the best approaches to mitigate the occurrence of risks [83]. According to Sánchez-Gordón and Colomo-Palacios [84], effective cooperation and sharing the responsibility of security between teams are considered to be the core of DevSecOps as software engineers need to work collaboratively on security and manage risk properly. They also pointed out the importance of automating security practices within DevOps pipelines in tandem with agility to achieve reliable security [90].

Another multivocal literature review (MLR) by Rafi et al. [85] provided a definition for data heterogeneity and described the key problem of integrated software environments that hinders data assessment and quality. They suggested implementing specialized adaptors to share artifacts that could be accessed from different domains throughout the development process. On the other side, they emphasized the obligation of understanding the data structure in heterogeneous environments. The same authors have conducted another SLR [86], where they classified DevSecOps principles (culture, automation, measurement, and sharing) and pointed out how disseminated environments make it difficult for security analysts to operate security practices from different perspectives (processing, execution, and testing). They stated problems that would lead to improper threat assessment, like inconsistent policy designs, poor performance measures, and lack of adequate channels to monitor team coordination during the development phase. In another SLR of [87], the authors discussed how some organizations started implementing DevSecOps practices without adequate preparation, resulting in massive failures in penetration testing. Software errors also happen due to a lack of configuration settings and ineffective management of operational inspection and user feedback.

Chiari et al. conducted a survey on static security analysis for IaC scripts [89]. The authors discussed failure criteria for performance analysis to identify inefficient configuration from monitored parameters. They also highlighted the benefits of applying static analysis in IaC scripts to let developers and admins automate applications without requiring expensive or time-consuming repair for isolated and/or defective infrastructures. Another SLR was published by Ramaj et al. in 2022 [88]. Their paper shed light on issues related to security compliance during the implementation of DevSecOps. The authors recommended that developers should consider security regulations and accessibility frameworks when building, testing, and deploying software solutions. Further, lack of security awareness is one of the major reasons that could hamper DevSecOps adoption and delay the implementation of security controls into CI/CD pipelines.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 | Comparison of existing literature reviews on security controls in DevSecOps with our SLR: A: Present the study about integrating security controls in DevSecOps? B: Present the study involving challenges of security controls within the adoption of DevSecOps? C: Present the study relating to security control–related solutions or workarounds? D: Present the study identify areas of gap analysis in security controls within the DevSecOps environment? (✓✓) The paper fully covers it. (✓) The paper partially covers it. (N/A) Not applicable.

Title	Focus	Year	A	B	C	D
Challenges and Solutions When Adopting DevSecOps: A Systematic Review [72]	(1) This SLR presents an overview of key studies to address research questions. (2) DevSecOps-related challenges are determined and categorized in aspects (people, practices, tools, and infrastructure). (3) Solutions are formulated along with the mapping with enlisted challenges based on what was found in primary studies.	2021	✓	✓	✓	✓
DevSecOps: A Multivocal Literature Review [83]	(1) This paper indicates how DevSecOps works effectively by integrating modern security methods. (2) It gives a comprehensive overview of how DevSecOps could possibly be implemented in a way that benefits from continuous and automated improvement practices. (3) The book describes the benefits of DevSecOps and the difficulties an organization might experience when operating DevSecOps.	2017	✓	✓	N/A	N/A
Security as Culture: A Systematic Literature Review of DevSecOps [84]	(1) This SLR focuses on the concept of collaboration and sharing the responsibility of security between teams. (2) It points out how DevSecOps allows the automation of security practices in an agile manner within DevOps pipelines. (3) DevSecOps requires a seamless security integration in tandem with velocity and reliability. (4) Effective collaboration on security is much needed as software engineers need to manage objectives and risk analysis.	2020	✓	N/A	N/A	N/A
Multicriteria Based Decision Making of DevOps Data Quality Assessment Challenges Using Fuzzy TOPSIS [85]	(1) Heterogeneity and diversity of components can results in problems during the assessment of data quality. (2) Integrating security controls across data components is another challenge. (3) Invalid data and missing information are major impediments to DevSecOps adoption.	2020	✓	✓	N/A	✓
Prioritization Based Taxonomy of DevOps Security Challenges Using PROMETHEE [86]	(1) This study focuses on security manual testing and configuration issues. (2) Threat modeling issues and lack of teamwork between security, development, and operation teams are addressed. (3) Developers may resist embedding security controls as it could slow down the deployment proceeding. (4) Inconsistent security policies and complex design templates were spotted. (5) Deficiency of testing measures and performance criteria used for security.	2020	✓	✓	N/A	✓
Toward Successful DevSecOps in Software Development Organizations: A Decision-Making Framework [87]	(1) Poor processes of penetration testing. (2) Inadequate deployment parameters and configuration settings used in security controls and compliance. (3) Issues in monitoring and tracking systems resulting in delayed feedback.	2022	✓	✓✓	N/A	N/A
Holding on to Compliance While Adopting DevSecOps: An SLR [88]	(1) Security compliance is assorted in three taxonomies: Initiation, management, and technicalities. (2) Security is reviewed as a nonfunctional process. (3) Different levels of security awareness can impact the pipeline activities. (4) Container images should be regularly scanned to detect vulnerabilities. (5) Compliance assessment is discussed regardless of the operation mode of executed security controls.	2022	✓	✓	N/A	N/A

(Continues)

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 | (Continued)

Title	Focus	Year	A	B	C	D
Static Analysis of Infrastructure as Code: A Survey [89]	(1) Insufficient infrastructure-as-code (IaC) patterns would lead to classification problems and bad-quality code associated with bugs and compilation errors. (2) Hardware machines require new data sets to represent diverse samples and parameters.	2022	✓	✓	N/A	N/A
This paper	(1) Challenges and major issues related to security controls during the adoption of DevSecOps. Those issues are classified according to ISO 27001:2022 Standards to make it easier for security practitioners to understand the reasons behind those problems and try to, avoid, or rectify them. (2) Introduce the solutions and workarounds as suggested in the literature and related to security controls trying to address the previously enlisted challenges. (3) Propose opportunities for future research/gap areas to support the integration of security controls in DevSecOps environments.	2024	✓✓	✓✓	✓✓	✓✓

3 | Research Methodology

SLRs are identified as one of the prevalent research methods in evidence-based software engineering (EBSE) for gathering data and analyzing pieces of evidences [91, 92]. Therefore, we decided to carry out an SLR to gain a deep understanding of security controls in DevSecOps following the methodology proposed by Kitchenham and Charters [93]. To this end, three research questions (RQs) are formulated (see Table 3). Following the guidelines and instructions in [91], this SLR is designed to present the following sections: (i) RQs, (ii) search strategy, (iii) inclusion–exclusion criteria, (iv) quality assessment criteria, (v) snowballing activity, (vi) data extraction, and (vii) data synthesis and mapping.

3.1 | Research Questions

We outlined the RQs and the motivation (purpose) behind each question in Table 3. Meanwhile, papers were searched and filtered. The process is described in the following sections.

3.2 | Research Strategy

With reference to the topic of this study and following the guidelines outlined by Kitchenham and Charters [97], our search string consists of three sections; the first section targets cybersecurity-related terms (e.g., security and cyber), the second part points to the most popular term of DevSecOps including other similar terms that are used interchangeably (e.g., SecDevOps and CyberDevOps). The last part of the search string extensively captures terms related to controls, including related terms (e.g., settings, standards, and guidelines). Eventually, all authors provided their input, reviewed the search string, and confirmed the final structure after a set of iterations as in Table 4.

3.3 | Digital Research Databases

Owing to the fact that there is a significant overlap among digital research databases, we chose five prevalent and widely used databases in software engineering and cybersecurity research (Scopus, IEEE Xplore, Science Direct, ACM Digital Library, and SpringerLink) to identify relevant papers for this study [98]. Next, pilot searches were conducted on the selected databases to confirm that the relevant papers were selected. Note that digital databases have unique syntax or search capabilities. Therefore, we customized the search string to use the full potential of the databases' search capabilities to get more relevant and accurate paper retrieval. Some databases may support regular expressions and/or Boolean connectors, while other libraries use specific commands for phrase searching. As an example, we customized the search string to match specific syntaxes like "continuous" and ensure better results.

3.4 | Inclusion and Exclusion Criteria

The following inclusion and exclusion criteria were applied to target relevant studies and remove irrelevant papers, duplicates, or any gray literature.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 3 | Research questions addressed in this study.

RQ	Description	Purpose/motive
RQ1	What challenges related to integrating security controls in DevSecOps are reported in the literature?	For those organizations who plan to adopt DevSecOps practices, security practitioners are advised to be aware of potential challenges concerning security controls and explore possible issues at the early stages of the SDLC [94, 95]. However, obstacles and impediments concerning security controls during the implementation of DevSecOps have not been precisely investigated. There are not enough peer-reviewed sources or significant bodies of up-to-date academic literature that thematically analyze and reconcile problems associated with the integration of security controls in DevSecOps. Hence, this research question aims to address this specific gap.
RQ2	What solutions are reported in the literature to address the issues associated with security controls in DevSecOps?	Security practitioners are interested in exploring and learning best practices, guidelines, and modern frameworks concerning the integration of security controls in DevSecOps [96]. This field has not been thoroughly discussed in the current DevSecOps research. Knowledge of what precautions/activities of security practitioners can solve and/or overcome security control–related issues at an early stage. Addressing this question helps developers learn from past DevSecOps experiences and scrutinize why some practices fail.
RQ3	What are the gap areas and/or opportunities for future research concerning security controls in the domain of DevSecOps?	This RQ aims to identify gaps and opportunities for future research. Further, we will explicitly highlight the shortcomings and areas where security controls lack and/or fail to operate properly while carrying out DevSecOps practices.

TABLE 4 | Research string used to extract relevant papers.

("security" OR "cyber" OR "cybersecurity") AND ("DevOps" OR "DevSecOps" OR "SecDevOps" OR "CyberDevOps") AND ("control" OR "standard" OR "governance" OR "compliance" OR "regulation" OR "audit" OR "safeguard" OR "measure" or "countermeasure" OR "assessment" OR "evaluation" OR "strategy" OR "appraisal" OR "criteria" OR "setting" OR "requirement" OR "logistic" OR "category" OR "controller" OR "mechanism" OR "instruction" OR "objective" OR "rule" OR "setup" OR "set-up" OR "regulate" OR "restriction").

Inclusion criteria

1. The article should specifically address the research objective or address one of the RQs.
2. The article should tackle one or more of DevOps or DevSecOps stages on top of integrating security controls in continuous security practices.
3. The article should discuss solutions or challenges that might be experienced while implementing and managing security controls in DevSecOps.
4. The article must be written in English and formatted as full-text articles.
5. The article must be published from 2012 to 2024 without restriction on publication place or venue.

Exclusion criteria

1. Articles that are not related to DevSecOps in cybersecurity or not peer-reviewed, like presentations or calls for papers (CFP).
2. Articles that do not provide information about challenges or solutions for handling security controls in DevSecOps.
3. Secondary studies (e.g., surveys and literature reviews).
4. Articles are written in any language other than English.
5. Articles cannot be accessed or downloaded.

3.5 | Data Selection

In conducting this SLR, the research team followed a systematic approach to select relevant papers to understand and ensure the reliability and accuracy of their study. Although the first author led the data selection process, it was a collaborative effort where all authors contributed to ensure the selection was accurate, thorough, and aligned with the study goals. It also helped to produce precise, consistent, and unbiased datasets. At the start, the first author drafted the inclusion and exclusion criteria based on the research objectives and methodology. Then, *First Author* collaborated with the co-authors to interpret and refine these criteria, ensuring they were comprehensive and consistent in a way all authors interpreted the criterion similarly. Afterward, *First Author* screened the articles based on the agreed criteria. *First Author* reviewed the titles and abstracts independently to confirm carrying out relevant articles and key studies to the next round. Later, *First Author* did full-text reviews to thoroughly investigate the papers resulting from the previous step. On the

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

other hand, the first author coordinated meetings to discuss other authors' perspectives and resolve disagreements, aiming to reach a final consensus. *First Author* conducted the final review and QA to ensure that all included papers met the review criteria and that no critical studies were missed. Throughout the study selection process, *First Author* kept essential documentation for decisions, criteria changes, and justifications. This documentation was shared with all co-authors to ensure transparency and allow them to include their supplementary materials as well.

Figure 1 shows the study selection process. In the first step, the search terms were initially used to search the database mentioned above sources in September 2023. The search terms matched only with the title, abstract, and keywords of papers. Table 5 shows how many papers were retrieved from the databases in each step. Initially, a total of 952 papers were retrieved; then, after removing the duplicates, 248 papers were left. Next, inclusion and exclusion criteria were applied to the titles, abstracts, or keywords of the 248 papers. Out of 248 papers, 138 were excluded in this step, and 110 were left for the next step. In the next step, the first author read the full content of the 110 papers and applied the inclusion and exclusion criteria. There were some instances where all the terms of the search string were clearly present in the title, abstract, or keywords. Nevertheless, it was difficult to decide whether the content of the article was genuinely related to the goals and RQs of this SLR. The research team used the guidelines outlined in [99] and conducted the following quality assessment checks to evaluate the papers. First, each paper was thoroughly examined to confirm that one or more of the RQs was being explicitly addressed in the paper. Secondly, we checked if the papers were adequately related to our SLR's focus (i.e., security in DevOps or DevSecOps or continuous practices). Thirdly, the papers had to sufficiently describe their findings, contributions, and contexts. As an example, the term "DevSecOps" was spotted in the titles or abstracts of some papers, yet after inspecting the whole paper, the content was not sufficiently related to DevOps or DevSecOps or even other security continuous practices. As shown in Figure 1, 41 papers met all inclusion and exclusion criteria. Selection decisions for each included or excluded paper were recorded in an Excel spreadsheet for each step. The papers in each step were labeled as Yes (i.e., the paper should be included for the next step), Maybe (i.e., not enough information available to exclude the paper in this step), or No (i.e., the paper should be excluded in this step). The spreadsheet was shared with other authors to thoroughly review the selected and excluded papers and the rationale behind the decisions in weekly meetings, considering appropriate contexts, findings, and stated claims of the studies until a final consensus was achieved about the selected papers.

3.6 | Snowballing Technique

With the aim to enhance the overall quality and depth of research, the first author conducted backward snowballing to explore and thoroughly dive into the references and citations in the initial primary papers (41), aiming to efficiently capture all related studies that were missed out in the initial search and to uncover the important papers that didn't appear in the previous section, as per the instructions and guidelines outlined by Wohlin et al. [100]. This technique is valuable for identifying additional relevant literature and crucial papers that could be useful for the study. The same inclusion and exclusion criteria were used to select more papers while ensuring a comprehensive and thorough literature review (see Figure 1). However, it was found that most of the papers were already included and captured in the initial selection process. As a result, only three more papers were added using this activity. Afterward, the snowballing technique was performed again on the reference list of the secondary studies mentioned in Table 2 to identify any potentially missed relevant papers. By screening and flipping through the reference list of the secondary studies, only one paper satisfied the inclusion/exclusion and quality assessment criteria. In conclusion,

TABLE 5 | Number of papers resulted from running Search Engine.

Databases	Step 1	Step 2	Step 3	Step 4
Scopus	321	130	39	23
IEEE Xplore	119	41	27	9
ACM Digital Library	52	26	22	4
SpringerLink	429	35	14	3
Science Direct	31	16	8	2
Total No. of papers	952	248	110	41

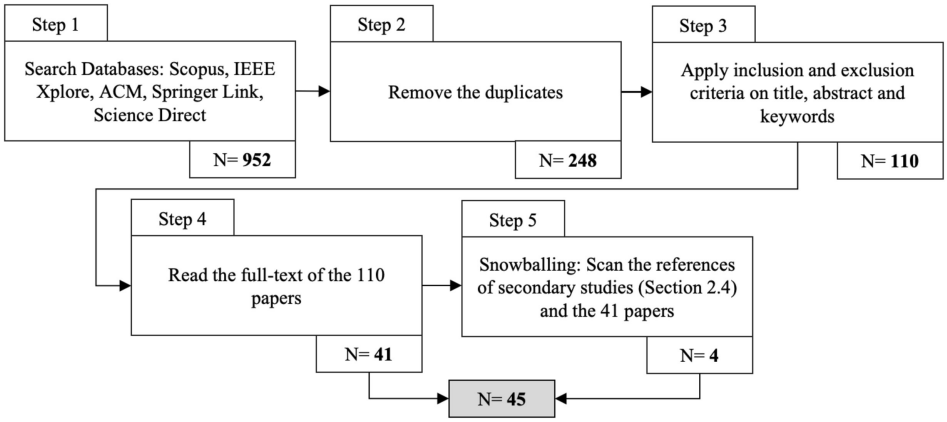

FIGURE 1 | Steps of the study selection process.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

the final count of in-scope (relevant) papers, for this study is 45 papers, as listed in Table 6.

3.7 | Data Extraction

This section outlines the process of collecting and recording relevant data from the 45 primary papers enlisted in Table 6. The first author followed the guidelines outlined by Garousi et al. [141] and Kitchenham [93]. *First Author* performed the data extraction process systematically into comprehensive standardized Excel spreadsheets involving the key attributes (e.g., title, keywords, and abstract) and corresponding data from the selected papers. Initially, *First Author* extracted data subsets individually (challenges, solutions, and research gaps) for each RQ. Next, *First Author* shared the completed spreadsheets with other authors to review the progress and to resolve discrepancies. *First Author* coordinated closely with the team to compare results and identify ambiguities in data interpretation or extraction. Any issues that arose, disagreements, and misinterpretations were discussed and resolved during those discussions. Later, *First Author* considered their input and feedback thoroughly to update the Excel sheets, ensuring the major aspects were covered and aligned with the RQs. This process helped to verify the consistency and accuracy of extracted data (results), ensuring reliability and thoroughness in tandem with overseeing the tracking progress.

3.8 | Data Synthesis and Analysis

The first author conducted thematic analysis [142] on the extracted data to analyze the results and address the RQs as suggested by Braun and Clarke [143]. This is a flexible and iterative process where researchers systematically revisit data, make adjustments, and uncover meaningful insights and interpretations that contribute to a better understanding of the data [144, 145]. In the first step, the first author reviewed the extracted data thoroughly and highlighted recurred and common patterns to get a broad context and divide it into three components according to RQs (challenges, solutions, and research gaps). Next, *First Author* reviewed the data in each component to gain familiarity and give a concise narration using descriptive key points. In each component, *First Author* revised the themes and created codes with appropriate names, to eliminate redundant themes, combine similar themes, or break down complex themes into subthemes. Next, the extracted themes were examined to confirm the credibility of the results (e.g., Is there enough evidence for each theme? Was it mentioned in at least two to three papers?) as suggested by Cruzes and Dyba [146]. Lastly, the themes and their corresponding codes were shared with the second and third authors for review. Then, in several meetings with all the authors, disagreements and conflicts on generated themes and codes were discussed, which were later resolved with the negotiated agreement approach [147].

As an example for analyzing one of the components (challenges), we classified the areas (domains in software engineering) that could be noticeably impacted by the enlisted challenges within building and integrating security controls in DevSecOps or might have the direct and biggest effect. This classification greatly helped to provide structured views for the challenges and address them effectively. Figure 2 presents an example of how the classification was formed based on the below impacted areas:

1. *Technical constraints*: Issues like system performance, agility, compatibility, limited or inadequate testing environments, or complex integrations of unfamiliar SW/HW components.
2. *Resource limitations*: Staffing, budget, and resource limitations that restrict building or maintaining robust security controls.
3. *Cultural resistance*: Challenges in getting user buy-in, resistance to adopting new security strategies, and cultural resistance within the organization.
4. *Knowledge and skill gaps*: Lack of experience on cyber threats and latest attacks, leading to difficulties in application designs.

This process led to identifying 19 challenges (see Table 9 in Section 4.1), 18 solutions (see Table 8 in Section 4.2), and 12 research gaps (see Section 5). Subsequently, *First Author* conducted the mapping process and linked the *19 challenges* with the *18 proposed solutions*. Once the initial version of the challenges–solutions mapping was created, it was shared with the other two authors for feedback. Their feedback helped identify conflicts, potential discrepancies, and ambiguities that were resolved later using the negotiated agreement approach [147]. Figure 3 shows the challenges–solutions mapping.

4 | Results and Discussion

This section presents the results of RQs listed in Table 3 based on 45 primary papers in Table 6.

4.1 | RQ1: Challenges Related to Security Controls in DevSecOps

This SLR identified a list of 19 challenges that have affected or may affect the implementation of security controls in DevSecOps (see Table 8). At the highest level, they were described, analyzed, and categorized according to the overarching classification outlined in the ISO 27001:2022 Standard, such as organizational challenges (e.g., budget constraints and lack of executive support), technical challenges (e.g., complexity of HW/SW components and legacy system compatibility), or people-related challenges (e.g., user resistance to integrate security controls, and training needs) [148], as shown in Table 7. Table 8 also demonstrates some descriptive key points for each challenge, the application type (general, mission critical, or safety critical) in which the challenge could occur and the papers that discussed and debated on that challenge.

4.1.1 | CH01: Heterogeneous SW/HW Components and Incompatible Data Formats

The current literature points out some technical challenges concerning the integration of heterogeneous scripts that require

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 6 | Primary papers investigated and used to conduct this SLR (J: journal; C: conference; BC: book chapter; W: workshop).

ID	Title	Citation
P1	Challenges and Opportunities of DevOps in Cyber-Physical Production Systems Engineering [101]	5
P2	DevOps Capabilities, Practices, and Challenges: Insights From a Case Study [102]	178
P3	Revisit Security in the Era of DevOps: An Evidence-Based Inquiry Into DevSecOps Industry [59]	1
P4	A DevOps Framework for Quality-Driven Self-Protection in Web Software Systems [103]	9
P5	Towards a Hypothetical Framework to Secure DevOps Adoption: Grounded Theory Approach [104]	20
P6	Continuous Compliance: Experiences, Challenges, and Opportunities [105]	10
P7	An Empirical Analysis of Practitioners' Perspectives on Security Tool Integration Into DevOps [106]	14
P8	Modelling Continuous Security: A Conceptual Model for Automated DevSecOps Using Open-Source Software Over Cloud (ADOC) [107]	71
P9	Integrating Security With DevSecOps: Techniques and Challenges [108]	31
P10	DevOps for Information Management Systems [109]	71
P11	DevSecOps Approach in Software Development Case Study: Public Company Logistic Agency [110]	2
P12	Mitigating an Oxymoron: Compliance in a DevOps Environment [111]	15
P13	Microservices: Architecture, Container, and Challenges [112]	35
P14	Continuous Software Engineering and Beyond: Trends and Challenges [113]	325
P15	BP: Security Concerns and Best Practices for Automation of Software Deployment Processes An Industrial Case Study [114]	32
P16	A Unified Framework for Automating Software Security Analysis in DevSecOps [115]	4
P17	Using a Semantic Knowledge Base to Improve the Management of Security Reports in Industrial DevOps Projects [116]	4
P18	Embracing IaC Through the DevSecOps Philosophy: Concepts, Challenges, and a Reference Framework [117]	14
P19	Prioritization Based Taxonomy of DevOps Security Challenges Using PROMETHEE [86]	38
P20	Toward Successful DevSecOps in Software Development Organisations: A Decision-Making Framework [87]	63
P21	Security Impacts of Sub-Optimal DevSecOps Implementations in a Highly Regulated Environment [118]	17
P22	An Advanced DevOps Environment for Microservice-Based Applications [119]	11
P23	Beyond Continuous Delivery: An Empirical Investigation of Continuous Deployment Challenges [120]	96
P24	Container and Microservice Driven Design for Cloud Infrastructure DevOps [121]	275
P25	The Top 10 Adages in Continuous Deployment [122]	107
P26	Continuous Delivery Huge Benefits, but Challenges Too [123]	485
P27	BP: Security Concerns and Best Practices for Automation of Software Deployment Processes: An Industrial Case Study [114]	32
P28	Hardening With Scapolite: A DevOps-Based Approach for Improved Authoring and Testing of Security-Configuration Guides in Large-Scale Organizations [124]	3
P29	Root-of-Trust for Continuous Integration and Continuous Deployment Pipeline in Cloud Computing [125]	0
P30	Efficient Secure DevOps Using Process Mining and Attack Defense Trees [126]	0
P31	Secure Scale: Exploring Synergies Between Security and Scalability in Software Development and Operation [127]	0
P32	Cloud Infrastructure Self Service Delivery System Using Infrastructure as Code [128]	2
P33	Migration From DevOps to DevSecOps: A Complete Migration Framework, Challenges, and Evaluation [129]	3
P34	Automated Dependability Assessment in DevOps Environments [130]	2

(Continues)

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 6 | (Continued)

ID	Title	Citation
P35	Challenges and Approaches for the Assessment of Micro-Service Architecture Deployment Alternatives in DevOps [131]	5
P36	From DevOps to DevSecOps Is Not Enough. CyberDevOps: An Extreme Shifting-Left Architecture to Bring Cybersecurity Within Software Security Lifecycle Pipeline [1]	2
P37	Major Challenges of Systems-of-Systems With Cloud and DevOps—A Financial Experience Report [132]	9
P38	Continuous Validation for Data Analytics Systems [133]	23
P39	Architecture Scheme of DevOps for Cross Network and Multiple Environment Collaboration [134]	2
P40	Understandings of Ethics in DevOps Teams: A Preliminary Investigation [135]	0
P41	Current and Future Challenges of Software Engineering for Services and Applications [136]	68
P42	Self-Service Cybersecurity Monitoring as Enabler for DevSecOps [137]	62
P43	Expanding DevSecOps Practices and Clarifying the Concepts Within Kubernetes Ecosystem [138]	3
P44	VeriDevOps: Automated Protection and Prevention to Meet Security Requirements in DevOps [139]	13
P45	Rethinking Secure DevOps Threat Modelling: The Need for a Dual Velocity Approach [140]	4

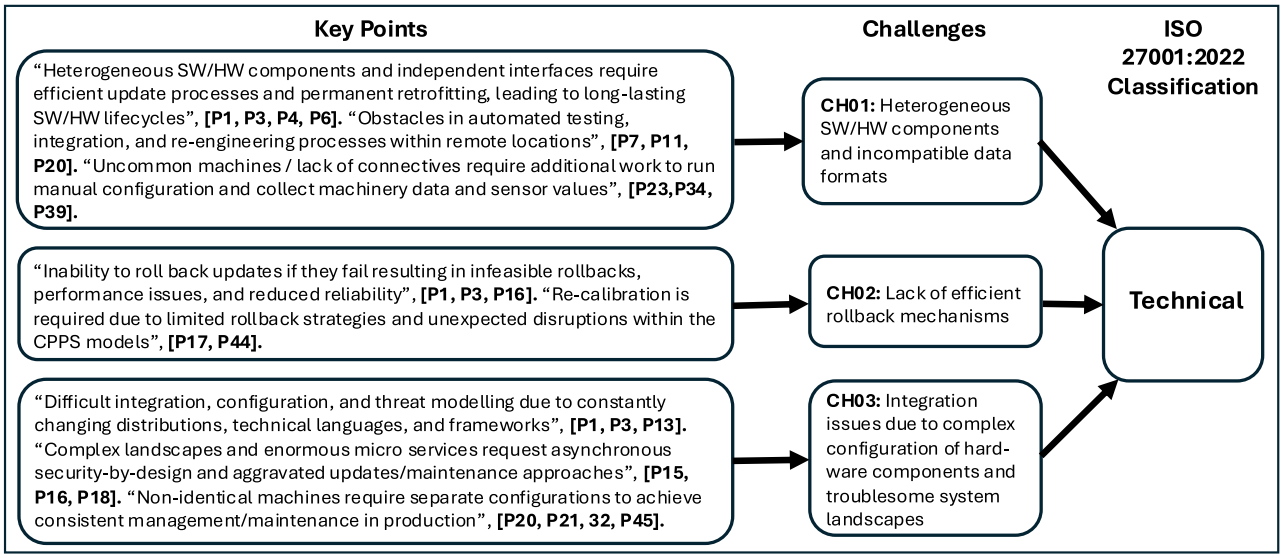

FIGURE 2 | Applying thematic analysis on a sample of collected data.

managing diverse components and endpoints (e.g., operating system resources, file permissions, and logging services) *[P3, P4]*. The tight coupling between software (SW) and hardware (HW) components from one side and regulatory environments from the other side, requires manual configurations and presents serious obstacles hindering the success of CD *[P1, P10]*. Furthermore, fixing the problem of inconsistent SW/HW segments and re-engineering the split (fragmentation) between incompatible components are considered troublesome. This resulted from complicated connectives and robust elements in remote locations within development and operation processes *[P20, P23, P26]*. Moreover, outdated security techniques are still used in software design and execution phases, resulting in a serious challenge of aligning with business interests *[P6]*. The main confusion comes from the increasingly sophisticated attacks that cannot be predicted by obsolete techniques at design and runtime phases *[P7, P11]*. On the other hand, providing adequate information and briefings on development purposes is still difficult due to the complexity and dissimilarity found in machines *[P34, P39]*.

4.1.2 | CH02: Lack of Efficient Rollback Mechanisms

The inability to roll back updates when they fail would significantly impact the overall performance and reduce software reliability. Furthermore, infeasible rollback mechanisms and limited reporting would not assist the development processes

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

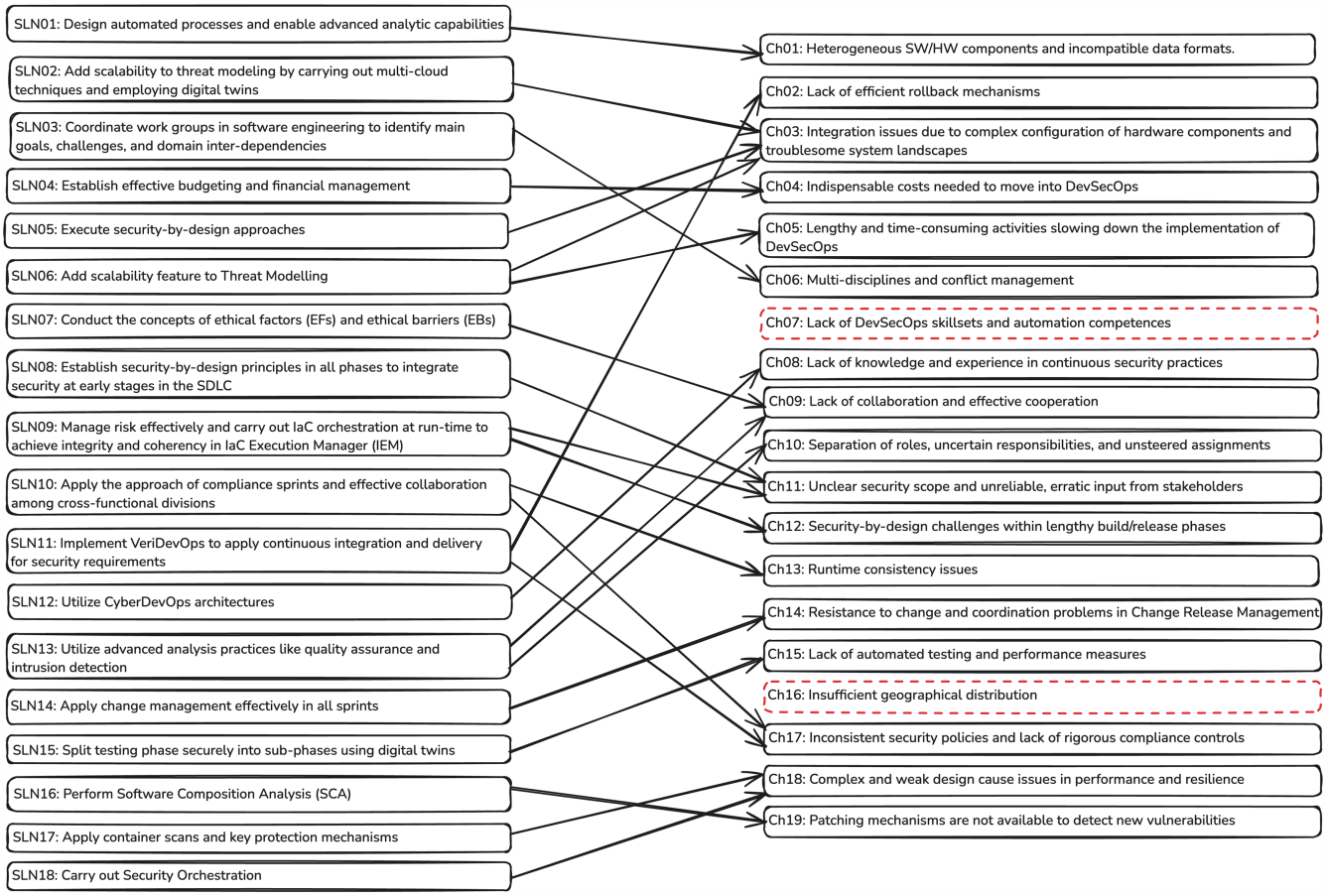

FIGURE 3 | Solutions (Table 9) that address the challenges related to security controls in DevSecOps (Table 8).

TABLE 7 | ISO 27001:2022 Classification.

Classification	Examples
Organizational	Policies, roles, responsibilities, and resources required to manage information security.
People	Security awareness, training, and human resource security.
Physical	Protection of physical assets such as buildings, servers, and workstations.
Technical	Access control, encryption, and malware protection to safeguard information systems.

when software updates fail to execute *[P1]*. Conversely, they will negatively impact both the performance and reliability of cyber-physical production systems (CPPS) engineering *[P3, P16]*. In comparison with common data environment (CDE) practices, CD requires sophisticated monitoring solutions and fully automated rollback mechanisms *[P23]*. Because the vast amount of data produced by security activities varies in formats, perspectives, and contents *[P17]*; thus, extra data processing and recalibration activities are certainly needed to turn out unexpected disruptions and downtime errors *[P44]*.

4.1.3 | CH03: Integration Issues due to Complex Configuration of Hardware Components and Troublesome System Landscapes

Many microservices with frequently changing distributions running on multiple nodes (e.g., physical or virtual machines) would result in complicated hardware modeling, tricky integration, and more communication disruptions *[P1]*. Nonidentical machines require separate configurations to achieve consistent modularization in production phases *[P3]*. Due to the reason that microservices are created and destroyed dynamically, there is a lack of correspondence and proper resemblance between microservices and associated system nodes *[P16]*. A bunch of asynchronous interactions is required to connect microservices with complicated call chains, resulting in adding complexity to system configurations, inconsistent environmental settings, and runtime failures as well *[P13, P15]*. On top of all, developers find it difficult and troublesome to understand evolving system components thoroughly (in-depth), especially with the increasing complexity of applications *[P18]*. This will impede and hinder the scalability of threat modeling. Therefore, threat modeling is considered a bottleneck due to the need for significant time to execute properly *[P20]*. In addition, diverse substructures and infrastructures make manual processes quite burdensome depending on the application phase (e.g., development, testing, integration, or production). They require complex integration and asynchronous practices to perform operational transactions

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 8 | Challenges related to security controls in DevSecOps adoption, along with the papers contributed to those challenges.

ISO 27001:2022 Classification	Challenge	Key points	App. types	Papers
Technical	CH01: Heterogeneous SW/HW components and incompatible data formats	• Heterogeneous SW/HW components and independent interfaces require efficient update processes and permanent retrofitting, leading to long-lasting SW/HW lifecycles. • Obstacles in automated testing, integration, and re-engineering processes within remote locations. • Uncommon machines and lack of connectives require additional work to run manual configuration and collect machinery data and sensor values. • More tasks are needed to identify security controls leaving developers with puffed workloads and incomprehensive documentation.	General	P1, P3, P4, P6, P7, P10, P11, P20, P23, P26, P34, P39
Technical	CH02: Lack of efficient rollback mechanisms	• Inability to roll back updates if they fail resulting in infeasible rollbacks, performance issues, and reduced reliability. • Recalibration is required due to limited rollback strategies and unexpected disruptions within the CPPS models.	Mission critical	P1, P3, P16, P17, P44
Technical	CH03: Integration issues due to complex configuration of hardware components and troublesome system landscapes	• Difficult integration, configuration, and threat modeling due to constantly changing distributions, technical languages, and frameworks. • Complex landscapes and enormous microservices request asynchronous security-by-design and aggravated updates/maintenance approaches. • Nonidentical machines require separate configurations to achieve consistent management/maintenance in production.	Mission critical	P1, P3, P13, P15, P16, P18, P20, P21, P32, P45
Organizational	CH04: Indispensable costs needed to move into DevSecOps	• Increased cost required for modularization, procurement, integration and variable tests. • Extra cost to fix disruptions and potential damage of machinery during production schedules. • Inability to spend shared funds effectively.	Mission critical	P1, P3, P41
Technical	CH05: Lengthy and time-consuming time-consuming activities slowing down the implementation of DevSecOps	• Time loss due to manual testing, code reviews, and delivery failures in mechanics functionalities. • Lack of capabilities and appropriately skilled staff would delay the development process. • Physical isolation of multiple architectures results in long development cycles.	General	P2, P3, P8, P13, P14, P15, P16, P21, P23, P25, P39, P41, P45

(Continues)

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 8 | (Continued)

ISO 27001:2022 Classification	Challenge	Key points	App. types	Papers
People	CH06: Multidisciplines and conflict management	• Difficult to unify individuals of different cultures and backgrounds to work in collaborative groups. • Unclear understanding of objectives, expectations, and limitations of system of systems (SoS) context. • Managing directors do not fully understand DevSecOps concepts. • Social issues between development and operation teams.	General	P1, P2, P3, P9, P10, P20, P24, P26, P33, P37, P40
People	CH07: Lack of DevSecOps skillsets and automation competencies	• Need to recruit staff with DevSecOps skills to cope with cyberattacks or retain the staff who have relevant security experience. • Teams are not ready to be on calls for operational problems because of not possessing automation skills. • The false perception of considering automation as the "bottleneck" of DevSecOps adoption.	General	P2, P3, P8, P16, P17, P21, P23, P27
People	CH08: Lack of knowledge and experience in continuous security practices	• Need to prepare workshops and high-quality training programs to keep developers updated with continuous security practices and trendy techniques. • Deal with stubborn or demotivated customers trying to change their mindsets and convince them to fully adopt continuous security practices.	General	P2, P9, P10, P23, P35
People	CH09: Lack of collaboration and effective cooperation	• Collaboration and transparency issues, complicating the involvement of security practitioners and putting DevSecOps progress at risk. • Poor techniques to monitor collaboration among team members.	General	P1, P2, P5, P25, P33, P40, P41, P43
People	CH10: Separation of roles, uncertain responsibilities, and unsteered assignments	• Unclear role definitions result in conflicts and misunderstandings about work responsibilities and assignments. • Vague tasks and assignments would lead to a high volume of staff turnovers, burnouts, and shifted responsibilities. • Unsteered assignments result in many challenges in knowledge sharing and expert acquaintance. • Slowing down the process of upskilling the staff and improving their cyber capabilities, as they would resist change or improve in such environments.	General	P1, P2, P3, P6, P10, P27

(Continues)

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 8 | (Continued)

ISO 27001:2022 Classification	Challenge	Key points	App. types	Papers
Organizational	CH11: Unclear security scope and unreliable, erratic input from stakeholders	• Unreliable and erratic input leads to isolated infrastructure-as-code (IaC) patterns and complicated data utilization. • Inconstant user feedback leads to poor risk analysis and destructive impacts on security. • Hard to know requirements and feedback upfront, hence, frequent data reviews are needed to assist security and decision-making.	General	P8, P14, P16, P17, P18, P20, P24, P30, P31, P35, P42
Technical	CH12: Security-by-design challenges within lengthy build/release phases	• Embedding security controls requires maintenance and safety prerequisites. • Lack of stable platforms and ever-evolving threat landscapes cause many issues impacting both security and productivity.	Safety critical	P1, P3, P21, P32, P39
Organizational	CH13: Runtime consistency issues	• Connectivity problems make it hard to prioritize security issues in remote infrastructures. • Customer systems are unable to adhere to security in tandem with agile.	Mission critical	P3, P33
Organizational	CH14: Resistance to change and coordination problems in change release management	• Ignore/overlook change management in security. • Conduct end-to-end system of systems (SOS) leads to resistance and slow adoption of DevSecOps. • Impossible to release new versions quickly and safely.	Mission critical	P1, P2, P5, P10, P37
Technical	CH15: Lack of automated testing and performance measures	• Extended landscapes need automatic testing due to digital twins created in isolated environments. • Potential issues may come up due to the lack of performance measures during manual security testing.	Mission critical	P1, P15, P23, P27, P28
Organizational	CH16: Insufficient geographical distribution	• Incorrect geographic distribution for dev/sec/ops teams. • Hard to achieve DevSecOps visions due to distinct objectives across teams.	General	P2, P3, P25, P41
Organizational	CH17: Inconsistent security policies and lack of rigorous compliance controls	• Lack of strict compliance and consistent code standards lead to mitigation problems. • Ethical issues in legal compliance lead to reduced integrity and weak privacy. • DevOps professionals ignore security controls and take shortcuts to speed up the delivery. • Complex and ambiguous design of compliance controls, along with repetitive security guidelines, create big security challenges.	General	P3, P8, P9, P12, P14, P18, P21, P26, P35, P39, P43

(Continues)

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 8 | (Continued)

ISO 27001:2022 Classification	Challenge	Key points	App. types	Papers
Technical	CH18: Complex and weak design cause issues in performance and resilience	• Nonfunctional requirements (NFRs) add complexity to security controls and negatively impact the scalability. • Performance issues of legacy resources complicate the DevOps folding due to security audit failures. • Security controls need to be scanned continuously within CI/CD pipes. • Discrepant design of policies and regulations leads to poor performance and issues in scalability assessment.	General	P1, P2, P3, P4, P5, P6, P8, P18, P27, P37, P43
Technical	CH19: Patching mechanisms are not available to detect new vulnerabilities	• Difficult to identify/fix bugs that appear within relocation processes. • Failed patches ending up with code of 43% errors.	Mission critical	P8, P28, P29, P43

[P21, P32]. Because there are many components in cloud infrastructures (e.g., database, network, and storage), where each component is instantiated in multiple instances to engage extra elements to properly function and manage infrastructures resources (e.g., multiple virtual routers) *[P45]*.

4.1.4 | CH04: Indispensable Costs Needed to Move Into DevSecOps

Increased costs are required for dynamic interfaces like short-term hardware modularization, ever-changing architectures, elements, and IaC patterns *[P1]*. Frequent software updates and security constraints must be included at the architectural level of software development, which in turn will increase both complexity and cost *[P3]*. Also, expansion costs are needed for other major activities, such as simulation, procurement, variable data integration, and improving quality objectives like memory caches for performance *[P22]*. In contrast, individuals will regularly request additional funds to fix data disruptions, backtracking issues, and potential damage to machinery during production schedules. Consequently, there will always be a challenge of not being able to spend shared funds among teams effectively *[P41]*.

4.1.5 | CH05: Lengthy and Time-Consuming Activities Slowing Down the Implementation of DevSecOps

DevSecOps adoption involves lengthy delays and heavy workloads to perform AST *[P2, P3]*. Moreover, manual inspections and security scans need extensive scanning time that significantly impacts the overall execution time of CI/CD pipelines as many false-positive results need to be fixed and rectified at the end of the scanning process *[P8, P13]*. Due to the massive number of false-positive delays within a software release, developers eventually lose trust in tools' abilities to give accurate and reliable visualization of the software development process *[P14, P16]*. Development schedule slippage and critical time issues will impede the fulfillment of the requirements leaving the organization in constant struggles with data streamlining between CI/CD pipelines. For that reason, the incapacity to achieve and sustain the desired performance in the development phase causes work delays *[P21, P23]*. Besides, multiple code-batching activities are needed for code reviews within short time intervals to keep up with technical complexities, resulting in unsatisfactory response times and slowing down the whole process *[P25, P41]*. Within the DevSecOps environment, CI/CD pipelines are mainly created for testing purposes and addressing issues in operation mode *[P2]*. Incorporating security practices simultaneously during three phases (development, testing, and production) might end up with counterproductive consequences as it could lower the developmental pace of applications in production mode *[P3]*. If developers are given access to the production environment to fix vulnerabilities and rectify security issues, then there will be significant time loss due to delivery failures and continuous security practices in DevSecOps pipelines. In such cases, software engineers and developers usually disable applications in production site *[P15, P21]*. On the other side, some security activities like threat modeling require highly skilled engineers to conduct detailed analysis which makes it hard to maintain the pace of software development and align with business needs at the same time *[P25]*.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 9 | Solutions found in literature and related to challenges of security controls in DevSecOps adoption.

Solution	Challenges	Solution description	Papers
SLN01	CH01	• Design automated processes and allow analytic capabilities by executing CPPS models. • Careful design for DevOps processes to consider compatibility and put up with CPPS complexity and diversity. • Ensure continuous and efficient functioning of practices while maintaining security without affecting the project timeline or productivity. • Enable automated testing specification and execution within DevSecOps. • The research team should closely work with development teams to control problems of composite structures.	P1, P6, P28, P39
SLN02	CH03	• Add scalability feature to threat modeling and carry out multicloud techniques and shadowing systems. • Employ digital twins to simulate and jump-start the automatization within the development and operation phases. • Perform HW/SW couplings and link the digital twins with actual data to optimize the assembly process. • Implement rigid chains that use hierarchy-limited file formats to reduce the need for integration. • Apply contract-based designs and extra rules to specify functional/nonfunctional constraints and identify potential interactions among components.	P1, P11, P32
SLN03	CH06	• Organize work groups to clearly identify main goals, challenges, and interdependencies. • Conduct focus groups to collect testimonies and provide clear documentation about user feedback with the aim of keeping knowledge vital and consistent. • Perform fast modeling and continuous feedback to adapt and monitor changes in production.	P37, P40
SLN04	CH04	• Conduct budget management for software development and hardware modularization.	P1, P3
SLN05	CH03	• Execute security-by-design procedures to identify destination locations and confirm they match host requirements. • Automate the IaC process from design to runtime. • Carry out metadata indicators (e.g., OPC-UA and MQTT) to collect machinery data and store it in heterogeneous repositories, taking into account the simulation analysis. • Implement DevSecOps techniques with borderlines to filter and authenticate data. • Enforce compliance scripts and utilize the "chef role" tool based on configuration settings on endpoints. • Apply subsequent monitoring and continuous compliance frameworks to detect abnormalities, support overrides, and customs requirements.	P5, P6, P18, P20, P37, P43
SLN06	CH03, CH05	• Establish unified frameworks to automate security integration (tools and procedures). • Generate integration artifacts into DevOps pipelines. • Execute blacklist definitions to deal with the negative effects of secure configurations. • Apply a set of artificial intelligence for IT operations (AI Ops) based on natural language processing (NLP) for application logs.	P16, P18, P24, P28, P45

(Continues)

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 9 | (Continued)

Solution	Challenges	Solution description	Papers
SLN07	CH09	• Use ethical factors (EFs) and ethical barriers (EBs) to address ethical issues on time. • Carry out regular reviews for development processes to identify areas where security needs to be fixed.	P40
SLN08	CH11	• Establish security-by-design principles to meet security requirements throughout the CPPS life cycle. • Utilize digital twins to apply testing in secure settings. • Establish automated integration and explore MBEs to support self-testing and rollback of failed updates. • Use model-based frameworks to address security challenges through focus groups.	P1, P16, P17, P35
SLN09	CH11, CH12	• Execute risk management competently to report results. • Automate user feedback to obtain input from stakeholders. • Run automated testing through different releases to perform cross-validation and achieve better integration and rollouts. • Carry out orchestration at runtime to achieve coherent IaC execution manager (IEM). • Deploy IaC tools like Canary Sandbox Environment (CSE) and BizDev to test dynamic properties.	P1, P14, P17, P18, P28, P38
SLN10	CH13, CH17	• Run compliance sprints for security standards to improve collaboration between agile and security teams. • Boost effective collaboration among developers, security practitioners, and the operations team.	P33, P35, P44
SLN11	CH02, CH17	• Implement VeriDevOps to apply continuous integration/delivery for security requirements. • Implement protection mechanisms (e.g., firewalls) and comply with security standards (e.g., ISA/IEC) during vulnerability detection. • Examine system designs to apply root-cause analysis for security breaches. • Identify remediation methods on code as patches/upgrades at the design level. • Use a parser within the knowledge base (KB) to apply repeated validation to monitor changes, identify contradictory information, and ensure a consistent KB deployment.	P16, P17, P44
SLN12	CH08	• Propose CyberDevOps architecture to integrate security within DevOps. • Revise composition software analysis to deal with security settings and involve vulnerability assessment.	P28, P36
SLN13	CH09, CH10	• Utilize advanced analysis practices like quality assurance and intrusion detection. • Enforce access control policies. • Establish mitigation processes that involve revoking invalid and unidentified access requests.	P1, P15
SLN14	CH14	• Apply change management to deliverable sprints. • Determine preapproved changes beforehand to achieve desired agility.	P37
SLN15	CH15	• Securely split the testing phase into subphases using the bridging twins in inner/outer CPPS loops. • Carry out regular user reviews (revalidation processes) to ensure all users have valid access and authentication.	P1, P15

(Continues)

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 9 | (Continued)

Solution	Challenges	Solution description	Papers
SLN16	CH19	• Perform software composition analysis (SCA) to identify vulnerabilities. • Implement extensible configuration mechanisms (XCCDF) to tailor applications. • Run trusted computing groups (TCG) and trusted computing technologies (TCT) to address issues through system-on-chip advancements. • Isolate services and protect the traffic overallocated access points.	P8, P28, P29, P43
SLN17	CH18	• Apply container scans and key protection mechanisms for data in all phases (storage, transit, and execution). • Conduct entity attestation (container, virtual machine, and hosting nodes).	P43
SLN18	CH18	• Perform security orchestration, automation, and response (SOAR) technologies during the implementation process to coordinate and automate practices within a single platform. • Apply application systems control and audit (ASCA) rules to ensure robust and reliable systems. • Apply two-level approval techniques to confirm that only relevant users can access applications and organizational data.	P16, P27

Conducting threat modeling for each application in each iteration is a slow process that could lead to slow implementations and increased risks of not achieving DevSecOps goals as well *[P39]*. Additionally, the physical isolation of multiple architects and lack of capabilities decelerate DevSecOps processes and result in long development lifecycles *[P45]*.

4.1.6 | CH06: Multidisciplines and Conflict Management

Effective collaboration and cooperation among individuals could be challenging as some developers find it difficult to work in a connected group rather than independently *[P1]*. People cannot be put together and expected to work due to the fact that they initially differ in cultures and skill sets *[P2]*. Based on surveys and consultation sessions held with different teams *(development, operation, and security)*, there could be concerns about how groups differ in their aspects (perspectives). For example, security analysts focus on advanced cybersecurity detection by inspecting traces and application log files, while developers focus on continuous monitoring and building up the code *[P3]*. Conflicts and disputes normally happen because development teams prefer fast pace coding and changing applications frequently, whereas security teams promote stability and resilience of applications against cyberattacks *[P9]*. It is exhausting to integrate security controls and adopt new methodologies after developers are already done with customizing applications and systems to suit customer requirements and business features *[P10]*. They find it hard to return to the development phase and fix errors where security was overlooked *[P19]*. Alternatively, team assembly and resource allocations in late stages of the project require deep analysis and staff assessment even with a mutual agreement to integrate security functions in the product and take into account the critical milestones of the project itself *[P20, P24]*. Each team has its own organizational interests, competing goals, work methods, and perceived areas of control. Hence, it is highly expected to observe resistance, stress, and other social issues among development and operation teams *[P26]*. Moreover, it becomes more challenging if there are no organizational guidelines to comply with in resolving issues and problems *[P33]*. Correspondingly, having many balls in the air would lead to disagreements and/or burnouts *[P37]*. Although DevSecOps benefits are generous and abundant, yet there is a bit of confusion with no consensus about cyber objectives as some existing literature published unclear information about security trendy techniques in DevSecOps *[P40]*.

4.1.7 | CH07: Lack of DevSecOps Skillsets and Automation Competences

The fundamental knowledge of when/where software practitioners should embed and integrate security controls into DevOps layers within the development phase is a significant necessity for employing automation and carrying out continuous security practices *[P3]*. Therefore, it has always been a huge challenge for organizations to provide high-quality training to employees and retain staff who have relevant skills and experience *[P16]*. This challenge relates to recruiting new individuals and retaining current staff *[P21]*. In some instances of unexpected incidents and operational problems, developers are still incapable of attending emergency and incident calls because they do not possess the required capabilities in automation, monitoring, and other continuous security practices *[P2, P8]*. Some companies still think of upskilling and enhancing the competencies of existing staff, mainly in automation, as the "bottleneck" of DevSecOps adoption *[P17, P23]*. Typically, security teams comprise data administrators who supervise database tables and manage access requests. They monitor daily transactions, report data statistics, and resolve

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

abnormalities and/or malicious activities *[P23]*. Nonetheless, the number of security analysts who are expert in DevSecOps is still low in the industry *[P27]*.

4.1.8 | CH08: Lack of Knowledge and Experience in Continuous Security Practices

The steep learning curve of software developers who need to learn advanced security techniques and gain new competences to work on new procedures is quite a big challenge. They are requested to know exactly when and where to utilize security practices and integrate them into CI/CD pipelines, requiring urgent assistance from cybersecurity teams *[P2]*. Lack of security capabilities and DevSecOps competencies can lead the inexperienced engineers take more time to explore security tools and become familiar with continuous security practices *[P9]*. Although developers are keen to learn emerging technologies, but the challenge is to arrange enough workshops and high-quality training to keep up with work demands and continuous security practices *[P10]*. Further, it was perceived that not all customers are mature and ready enough to accept continuous software releases *[P23]*. Dealing with demotivated customers, trying to make them change their mindsets to fully adopt DevSecOps and support automated testing practices, it is considered a major work burden *[P35]*.

4.1.9 | CH09: Lack of Collaboration and Effective Cooperation

Adopting DevSecOps requires software engineers to familiarize themselves with security teams (e.g., risk and IR) and cooperatively work against cyberattacks and uncertainties *[P1, P2]*. Poor techniques and inadequate channels used to monitor teamwork and collaboration among divisions will result in serious communication issues hindering the implementation of DevSecOps *[P5, P19, P22, P25]*. DevSecOps execution is marked in danger whenever issues in transparency and collaborative culture rise up *[P33, P40, P41, P43]*.

4.1.10 | CH10: Separation of Roles, Uncertain Responsibilities, and Unsteered Assignments

Staff burnouts, shifted responsibilities, and segregation lines of business make the management of security controls tricky and challenging. Some business divisions may not interact efficiently, and instead, they will resist the adoption of continuous practices and new security methods, which will eventually result in slow upskilling and slack training *[P1]*. Moreover, lack of role definitions and unclear responsibilities result in organizational challenges and conflicts among individuals *[P2, P3]*. Changing responsibilities and turnovers caused by DevSecOps adoption can lead to misunderstandings about who is in charge of carrying out continuous security practices *[P6]*, leaving the organization in need to re-engineer the organizational structure of security roles along with associated responsibilities cohesively and rationally *[P10, P27]*. Consequently, sharing of knowledge between teams and business divisions would relatively become daunting and exhausting *[P10]*.

4.1.11 | CH11: Unclear Security Scope and Unreliable, Erratic Input From Stakeholders

Within the shift-to-left paradigm and pushing security to the left of the SDLC, many changes are pointed out by the QA team and release managers requiring to start with incomplete parameters *[P8]*. Unreliable input and delayed user feedback will eventually lead to inadequate risk analysis and poor workarounds without the ability to calculate the impacts of sudden attacks. Such risk issues could end up with isolation and destructive impacts on overall security and result in not achieving DevSecOps main objectives *[P14]*. Implementing DevSecOps practices requires applying IaC patterns, which make it difficult to utilize data and switch between software versions due to unexpected changes in security requirements. However, it is still hard to confirm the validity of collected data and determine whether the acquired security approaches were worth the cost or not *[P16, P17]*. Starting the requirement phase with incomplete input and vague information leads to time-consuming activities and difficulties in replicating errors as well *[P18, P20]*. In addition, security domain constraints and strict controls require further risk analysis and extra verification proceedings before each release, which can result in dividing the process into lengthy contingency plans and slowing down the execution of CPPS and other automatic security practices. Further, a large number of complicated software components require enormous prototype builds *[P24, P30, P31]*. Therefore, software engineers try to rely on manual quality checks and consult with stakeholders to clarify the scope and requirements to particularly operate in DevSecOps context *[P35, P42]*.

4.1.12 | CH12: Security-by-Design Challenges Within Lengthy Build/Release Phases

The security-by-design model embodies a strategic approach to building digital applications with reliable security, emphasizing threat modeling, risk assessment, and integrating preemptive security measures within every stage in the SDLC from inception to deployment *[P1]*. Security-by-design adheres to a set of rules and principles shaping up a core philosophy of security that includes prioritizing prevention over reaction and embodying a culture of continuous monitoring and improvement *[P3]*. One challenge of security by design is the misconception that security can be postponed, plus, retrofitting security controls onto existing systems is complex and costly, requiring contemporary maintenance and safety prerequisites *[P32]*. Also, evolving threat landscapes and current IaC patterns based on different tools and paradigms require continuous vigilance and adaptation *[P21]*. Addressing these challenges is integral to successfully implementing security-by-design principles and ensuring the long-term security of digital systems *[P39]*.

4.1.13 | CH13: Runtime Consistency Issues

Connectivity problems at runtime make it hard to prioritize security issues in remote infrastructures, while customers' old-fashioned techniques do not have the ability to adhere to security in tandem with agile *[P1]*. Such dependencies are numerous and restrictive, thus making it almost impossible to adopt DevSecOps simultaneously with agility and speed *[P2,*

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

P3]. To maintain both agility and speed of DevSecOps adoption, collaboration is required between developers, operations teams, and security professionals *[P22]*. Current security tools also need to be embedded and incorporated within DevOps layers in many instances of the SDLC *[P24]*. Moreover, runtime naming conventions in paths and configuration files, like including special characters, lead to failure in the CI/CD execution and, consequently, more difficulties during security bug fixes *[P33]*.

4.1.14 | CH14: Resistance to Change and Coordination Problems in Change Release Management

Working in the end-to-end system of systems (SoS) environments leads to resistance, impediments, and slow execution of DevSecOps practices *[P1, P2, P5]*. Some individuals may resist the integration process of security controls and/or ignore following up with the change management of product releases *[P10, P19, P37]*.

4.1.15 | CH15: Lack of Automated Testing and Performance Measures

Complex software environments need rigorous simulations and automatic testing due to the digital twins created in isolated environments. More serious issues and failures in user acceptance tests are prone to happen because of the absence of automatic testing for software applications before being delivered to the customer site *[P1]*. Research studies indicate that a large number of software iterations is spent on running regression tests and interpreting results of manual checks *[P15]*. Human intervention and assessment involved in user acceptance tests could bring more value and increase confidence in code quality *[P27]*. Lack of access control and poor simulation will result in simulation issues, thus worsening the deployment process and bringing potential changes on a regular basis *[P23]*. Security requirements must be checked and tested before delivering applications in production environments. In the case of indirect provisioning for access requests, developers reach out to operation teams to get a formal confirmation for each release at the customer site *[P28]*.

4.1.16 | CH16: Insufficient Geographical Distribution

DevSecOps concepts are not fully understood by the managing boards; therefore, developing efficient organizational structures for the teams is difficult *[P2, P3]*. Dysfunctional geographic locations for developers, security, and operations teams can result in poor and unsatisfactory communication *[P25, P41]*.

4.1.17 | CH17: Inconsistent Security Policies and Lack of Rigorous Compliance Controls

Lack of rigid security controls and coding standards leads to serious problems like delivering unreliable applications *[P3]*. In traditional processes, security compliance regimes usually restrict access to production servers to developers and testers only, which in turn hinders the process of CD *[P8, P9]*. Some software engineers may ignore the controls of security compliance and take shortcuts to speed up the development process, leaving their systems open to exploitation *[P21, P26]*. They believe implementing security measures toward the end of development is another challenging milestone. Thus, they continually try to overlook and pass over *[P35, P39]*. Furthermore, there are incompatible problems about how the sprint planning is negatively affected rather than ensuring security compliance *[P12, P14]*. Therefore, new roles and artifacts must be added to the project scrum to support complying with security compliance and regulations at the end of each sprint *[P18]*. On the other side, ethical issues and lack of collaboration increase the risk of inefficient security compliance. Also, some regulations and security measures can be overlooked instead of being thoroughly integrated within the SDLC *[P43]*.

4.1.18 | CH18: Complex and Weak Design Cause Issues in Performance and Resilience

Overall security posture cannot be improved nor rectified with ambiguous designs or inconsistent requirements *[P1, P2]*. Discrepant design of policies and performance measures, along with inconsistent security strategies and guidelines, will result in serious mitigation problems *[P5, P6]*. For those instances related to access provisioning, keeping inactive users leads to serious problems in mitigation. Therefore, access to systems and applications should be revoked automatically for invalid and unidentified users *[P18, P27]*. There are different security insights and perspectives about access requests during the mitigation process due to the fact that inactive users are not removed immediately *[P37, P43]*. Additionally, nonfunctional requirements (NFRs) of legacy systems create complications in security controls during their integration with DevSecOps practices. They require continuous scanning with several tools within the CI/CD pipelines *[P3, P4, P8]*, thus extending the CI execution time and increasing the complexity of configuring other continuous processes.

4.1.19 | CH19: Patching Mechanisms Are Not Available to Detect New Vulnerabilities

In security content automation protocols (SCAP), extensible configuration methods are used to define data inputs using machine-formatted checklists in open vulnerability and assessment languages *[P8, P28]*. Numerous vulnerabilities that cannot be identified, prioritized, or fixed may appear within relocation processes and running system patches *[P29, P43]*.

4.2 | RQ2: Solutions Concerning the Challenges Related to Security Controls in DevSecOps

This section presents a list of solutions found in the literature to address the security control-related challenges within the DevSecOps environment (see Table 9).

4.2.1 | SLN01: Design Automated Processes and Enable Advanced Analytic Capabilities

Automating processes is vital to ensure virtual machine compliance with security policies, providing flexible customization, and the ability to override when required. This is carried out by

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

executing CPPS, which utilize analytic capabilities and link conceptual models to store transformed data sources in wider repositories *[P1]*. Security practitioners must thoroughly work on designing DevSecOps processes and make them compatible with complex CPPS models. They aim to automate testing, maintain stability, and achieve continuous and efficient functioning of security practices *[P28]*. Furthermore, developers have started to integrate several technologies like the Internet of Things (IoT), data analytics, and metadata storage in accordance with typical objectives of machine learning (ML), application programming interface (API) integration, and data processing *[P6]*. In addition, researchers must work in parallel with development teams to discuss security problems that could arise within complex structures *[P39]*.

4.2.2 | SLN02: Add Scalability to Threat Modeling by Carrying Out Multicloud Techniques and Employing Digital Twins

Scalability is essential to ensure that threat modeling adapts to changing requirements, complexity, and the evolving nature of software systems. To achieve this, software engineers seek continuous feedback, perform CI/CD, and reuse threat libraries through different releases to apply and monitor changes in the production site. First, they set clear scope and security objectives for threat modeling efforts and use structured methodologies to document results. Subsequently, they leverage automation and prioritize threats based on their impacts. This is called integrating couplings of threat models into CI/CD pipelines to incorporate security in design and development phases *[P1]*. In addition, software engineers are advised to apply digital twins in pairs and link them with simulation tools and real-time data sources to optimize the assembly process and skip restrictive/slow routines. Digital twins perform continuous monitoring of virtual physical prototypes. In contrast, the simulation tools are crucial for improving resolution times and mitigating risks, as well as achieving the appropriate planning in various phases and preventing financial breakdowns during the SDLC. Another solution is to use contract-based designs to continuously specify functional and/or nonfunctional constraints in the design and runtime phases. In these instances, security analysts apply extra rules among services to define potential interactions and behaviors *[P11]*. Another solution is to implement multicloud methods and integrate tool chains with hierarchy-restricted file formats. Multicloud methods help operate and manage multiple cloud computing environments simultaneously. Their primary goal is to enhance flexibility, improve overall cloud security, and reduce vendor lock-ins, aiming to optimize application management and resource allocation. On the other side, involving flexible tool chains offers several benefits like improved consistency, easy adoption, efficient compliance, and long-term stability *[P32]*.

4.2.3 | SLN03: Coordinate Work Groups in Software Engineering to Identify Main Goals, Challenges, and Domain Interdependencies

Cross-functional teams to work together and collaboratively brainstorm to provide various benefits to the development process and software engineering as a whole, such as knowledge sharing, diverse perspectives, and accountability *[P37]*. The main goal is to conduct focus groups that work in tandem with team-related ethics to automate user feedback, collect essential documentation, and improve their learning to keep knowledge alive and consistent with reality *[P40]*.

4.2.4 | SLN04: Establish Effective Budgeting and Financial Management

Engineers are requested to develop comprehensive financial plans and allocate budget portions to security controls, continuous practices, SW/HW modularization, and other security expenses *[P1]*. They can regularly review and adjust budgets to fulfill security and financial objectives in accordance with changing circumstances *[P3]*.

4.2.5 | SLN05: Execute Security-by-Design Approaches

These approaches are essential for building secure and resilient applications in complex landscapes. Destination locations are predefined and verified to match host services requirements *[P37]*. By integrating security into every phase of the SDLC, organizations can reduce risks of security breaches and minimize potential impacts of incidents *[P5]*. Another solution is to automate IaC processes from design to runtime by carrying out metadata indicators (e.g., OPC-UA and MQTT). They are used to collect/store data in heterogeneous data lakes (repositories), taking into account simulation analysis *[P20]*. Developers can start using effective tools like "Chef Roles" to assign configuration endpoints and utilize compliance scripts *[P6]*. Additionally, research suggests continuous compliance frameworks where security compliance rules are managed and automated in a flexible and timely manner. Continuous compliance aims to support overrides, customize requirements, and provide real-time reports on security posture *[P18]*. Furthermore, security practitioners are recommended to apply subsequent monitoring to detect abnormalities and react at early stages *[P43]*.

4.2.6 | SLN06: Add Scalability Feature to Threat Modeling

Scalability is essential to help companies assess/mitigate security risks in an efficient manner *[P16]*. Integrating scalable frameworks within automated practices enables developers to better address security risks associated with application complexity *[P18, P24]*. This proactive approach builds resilient systems that react against security threats as they expand and evolve *[P28]*. On the other side, security practitioners conduct threat modeling to address scalability concerns like STRIDE, DREAD, CAPEC, and artificial intelligence (AI) for IT operations (AI Ops) that utilize natural language processing (NLP) to review blacklist definitions and deal with the negative effects of configurations *[P45]*.

4.2.7 | SLN07: Conduct the Concepts of Ethical Factors (EFs) and Ethical Barriers (EBs)

Developers must understand deployment cycles along with associated challenges. Similarly, deployment teams must understand SDLC-related obstacles. Both teams should be attentive to prevalent security attacks and the consequences of not considering

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

security controls in design phases. They are advised to carry out regular reviews and development assessments with EFs and EBs to locate where security should be rectified and improved *[P40]*.

4.2.8 | SLN08: Establish Security-by-Design Principles in All Phases to Integrate Security at Early Stages in the SDLC

In the light of DevSecOps deployment, the aim is to build a solid security foundation and minimize vulnerabilities from the very beginning throughout the development process *[P1]*. Executives prioritize security requirements from top to bottom and simultaneously educate developers on security up-to-date practices *[P16]*. They will start using model-based frameworks and digital twins to test settings and address security challenges through workshops and group sessions *[P17]*. They conduct automated practices (e.g., integration, testing, and virtual deployment) to explore model-based engineering (MBE) and fix rollback failures *[P35]*.

4.2.9 | SLN09: Manage Risk Effectively and Carry Out IaC Orchestration at Runtime to Achieve Integrity and Coherency in IaC Execution Manager (IEM)

Orchestration is crucial to configure properly and provision IaC patterns, thus working seamlessly to support the whole system *[P17]*. It refers to aligning IaC patterns with configuration and deployment parameters, then including infrastructure states in software codes (server, storage, network) to manage resource provisioning and access requests subsequently *[P18]*. On the other side, software engineers are advised to automate user feedback and execute risk management efficiently. In risk management, software engineers use structured frameworks to identify, assess, and mitigate risks while ensuring compliance with regulatory and internal governance requirements. They later report results in log files and await to obtain input from stakeholders via automated user feedback. This step would significantly reduce the need for periodic data collection efforts *[P1]*. Automation allows real-time feedback collection and enables organizations to respond quickly to cyberattacks and emerging issues. Generally speaking, automated systems ensure that data collection and feedback analysis are carried out consistently and efficiently. This promotes timely decisions and improves data consistency and accuracy, reducing errors and bias associated with manual processes *[P14]*. Another solution suggests running automatic testing before different releases to perform cross-validation and achieve better integration and rollouts *[P18]*. In modern software engineering, automatic testing is an essential component of CI/CD pipelines for enabling teams to test code changes automatically once integrated into databases and ensuring reliable delivery to production *[P28]*. Engineers deploy IaC scripts like Canary Sandbox Environment (CSE) and BizDev to conduct dynamic testing *[P38]*.

4.2.10 | SLN10: Apply the Approach of Compliance Sprints and Effective Collaboration Among Cross-Functional Divisions

Compliance sprints refer to a new initiative within the organization aiming to accelerate and prioritize compliance contexts during limited time frames. This examination involves evaluating processes, controls, and basic documentation to identify security cracks and areas of noncompliance *[P35]*. Some organizations run compliance sprints to quickly assess their current compliance regulations with specific criteria. They also involve intensive training programs and communication campaigns to educate engineers about compliance requirements and best security practices. Furthermore, if significant compliance violations are spotted, they initiate a compliance sprint immediately to investigate incidents, conduct root-cause analysis, and take corrective actions to prevent further violations *[P44]*. In DevSecOps, effective collaboration is crucial and should be complemented with automation and other practices like continuous testing and threat modeling. Therefore, all organizations are advised to improve collaboration and effective cooperation among agile and security teams. By doing this, they eventually build a robust security posture, and cross-functional divisions will cooperate and work together, ensuring security is handled as a shared responsibility *[P33]*.

4.2.11 | SLN11: Implement VeriDevOps to Apply CI and CD for Security Requirements

The main concept of VeriDevOps is to comply with security standards (e.g., ISA and IEC) by applying root-cause analysis for security breaches and implementing protection mechanisms (e.g., network firewalls and vulnerability detection) *[P16]*. Security professionals should reconfigure intelligent traffic monitors to avoid immediate danger and ensure system perimeters are consistent *[P17]*. They execute remediation methods on code and design upgrades using parsers within the knowledge base (KB) *[P38]*. Patching parsers are used to identify external human input (e.g., false-positive identification), duplicate findings, and KB changes. This allows analysts to revise and recalculate contradictory information, ensuring consistent deployments *[P44]*.

4.2.12 | SLN12: Utilize CyberDevOps Architectures

CyberDevOps involves integrating security into DevOps layers following hypothetical components like Security as Code, IaC, RBAC, IRM, and SETA methods *[P28]*. CyberDevOps frameworks allow developers to redo composite analysis and carry out vulnerability assessments *[P36]*.

4.2.13 | SLN13: Utilize Advanced Analysis Practices Like QA and Intrusion Detection

In cybersecurity, advanced analysis is used to detect vulnerabilities, respond to cyberattacks, and protect systems and networks. Further inspections often involve sophisticated software tools, statistical methods, programming languages like Python, and ML algorithms to process and analyze large and complex databases. The main purpose is to extract actionable insights, make data-driven decisions, and solve complex problems before they get harder or more expensive to fix *[P1]*. QA and intrusion detection are prevalent analysis practices used to ensure both the quality and reliability of software. QA activities prevent defects/issues from occurring and ensure that delivered products meet customer expectations and predefined quality standards. Network intrusion detection

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

(NID) refers to distinguishing malicious activities and responding to unauthorized transactions within networks. Enforcing NID practices and access requests control is crucial as it helps organizations protect their digital assets and revoke invalid and/or unidentified access requests. They detect and alert abnormalities that might compromise the security triad of data (CIA) *[P15]*.

4.2.14 | SLN14: Apply Change Management Effectively in All Sprints

Change management helps address concerns and resistance points of stakeholders to obtain consent and buy-ins. Managing changes emphasizes transparent team communication to reduce confusion and/or misunderstandings. This will also ensure that employees are in the loop and have adequate information about changes to start preparing with essential resources and training if needed. Ultimately, change management speeds up adopting DevSecOps processes as it builds capabilities to adapt to applied changes and seize new opportunities quickly *[P37]*.

4.2.15 | SLN15: Split Testing Phase Securely Into Subphases Using Digital Twins

Digital twins are consolidated within inner/outer CPPS loops to bridge phases and align data in comprehensive views, resulting in improving integration and synchronization *[P1]*. Further, software engineers conduct regular reviews and validation processes on current user names and identifiers (IDs) to ensure they have valid authentication and data access. This endorsement process is important to confirm current users remain accurate, up to date, and compliant with security standards and organizational regulations *[P15]*.

4.2.16 | SLN16: Perform Software Composition Analysis (SCA)

In modern software engineering, SCA is an important practice that uses third-party components extensively. It helps developers balance the leveraged benefits of open-source software and associated risks. Eventually, SCA helps organizations handle and mitigate risks associated with supply chains as their software heavily depends on open-source and third-party components *[P8]*. On the other hand, extensible configuration mechanisms (XCCDF) are suggested to incorporate new features and/or functionalities without altering core databases *[P28]*. Applying XCCDF and plug-in extensions improves modularization, scalability, and flexibility of system configuration *[P29]*. Format checklists and system-on-chip elements are used in trusted computing technologies (TCT) and open languages to isolate services before addressing security issues in the attempt to protect traffic over different access points *[P43]*.

4.2.17 | SLN17: Apply Container Scans and Key Protection Mechanisms

Research has suggested modern state-of-the-art approaches like applying container scans, key protection mechanisms, and entity attestations (container, virtual machine, and hosting nodes). Data will be scanned and processed in all phases within the SDLC (storage, transit, and execution) *[P43]*. DevSecOps engineers are encouraged to obtain operational data by utilizing application performance management (APM) and leverage on technologies in terms of speed and accuracy to achieve better productivity.

4.2.18 | SLN18: Carry Out Security Orchestration

Security orchestration is a cybersecurity practice that includes automation and integrating security practices to respond to and mitigate cyberattacks efficiently. The security orchestration, automation, and response (SOAR) approach aims to streamline IR processes and reduce the time required for detecting, investigating, and remediating attacks. Further, the SOAR approach enables effective team coordination and repetitive automation for tasks within a single platform *[P16]*. Security orchestration enhances the organization's ability to respond to security incidents in a rapid and coordinated manner, prioritize incidents, and reduce the risk of human error. Software engineers must comply with application systems control and audit (ASCA) guidelines every 6 months to ensure that systems are robust and reliable, in addition to adherence with two-level approval techniques which confirm that only relevant users get access to systems and organizational data *[P27]*.

5 | Discussion, Research Gaps, and Future Directions

Research gaps play a crucial role in driving the research progress and advancement of knowledge about security controls in the DevSecOps domain. They also reveal areas where further investigation is needed. This section discusses gap areas for future research concerning security controls in DevSecOps using the results (findings) of *RQ1* and *RQ2* clearly to address *RQ3* for detailed answer for research gap areas and future opportunities in DevSecOps Research. The mapping of the SLR results (Table 9) is provided to enable the reader to decide the connection between reported challenges, proposed solutions, and research gap areas comprehensively.

5.1 | Continuous Compliance Monitoring (CCM)

The findings of this SLR show a growing emphasis on CCM as a critical component of efficient security control management. Security controls are usually implemented to ensure systems are reliable, secure, and compliant with regulatory requirements. However, the static nature of traditional compliance assessments fails to address the dynamic and evolving threat landscape. This gap underscores the significance of CCM [149]. CCM facilitates real-time evaluation of security controls, ensuring they remain effective and compliant across systems and dynamic IT environments. By automating the monitoring process, CCM provides continuous visibility into the state of security controls, helping organizations identify and address deviations before they expand and escalate into significant risks [150]. For instance, tools such as SIEM platforms and compliance automation solutions are pivotal in implementing CCM by integrating compliance checks into daily operations [151].

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

CCM plays a vital role in ensuring that organizations meet regulatory requirements, internal security standards, and industry-specific requirements on an ongoing basis, using automation and real-time monitoring tools. The importance of CCM spans various dimensions, including risk management, regulatory adherence, protecting organizational reputation, and enhancing business agility [152]. Several studies report the challenges concerning security compliance and adherence with regulatory requirements *[CH17, CH18]*. Consequently, the following research gaps persist in this area and revolve around technological, methodological, and regulatory challenges:

- Interchange of compliance tools: More research is needed on how standardized frameworks can be developed for better interoperability between different compliance monitoring systems.
- Scalability and performance issues: More research is required to develop scalable components that can handle large-scale and real-time compliance monitoring across distributed systems.
- Integration of AI/ML for predictive compliance: More research is needed on how AI/ML models can be trained to predict compliance failures before they happen, to reduce risk exposure.
- Regulatory alignment: Organizations face increasing pressure to comply with standards such as ISO 27001, NIST CSF, and GDPR. CCM ensures continuous alignment with these requirements, reducing the risk of fines and penalties. However, there's a need to study automated mechanisms to update and adapt CCM systems as regulations change, without requiring costly manual updates.
- Resilience and adaptability: Further investigation is required to inspect the adaptive CCM mechanisms that could quickly adjust to changing environments without introducing extra security risks.

Despite the benefits, implementing CCM is not without challenges. Several studies highlight the complexity of integrating CCM tools into existing security systems, particularly in hybrid or multicloud environments [153, 154]. Additionally, there is a need for standardized metrics to measure the effectiveness of CCM in managing security controls. Future research should focus on developing frameworks that address these challenges, emphasizing interoperability, automation, and scalability [155].

5.2 | Automated Security Testing (AST)

Our SLR underscores the increasing adoption of AST as a crucial practice for ensuring the robustness and effectiveness of security controls. AST refers to using scripts and tools to identify vulnerabilities, validate security controls, and ensure that systems meet security standards and compliance requirements in an efficient and scalable manner [156]. Compared to manual testing practices, AST significantly enhances both the speed and accuracy of the testing process [157].

AST is significant to ensuring robust cybersecurity in several ways. For example, AST practices can consistently scan complex architectures, interconnected components, and third-party integrations in modern software systems, as they identify vulnerabilities faster and more efficiently [158]. In addition, automated testing provides immediate feedback to developers during the development process, allowing them to fix vulnerabilities as they are introduced. This real-time feedback is crucial to maintain security throughout the SDLC [159]. Manual testing can sometimes miss critical issues due to human fatigue, especially in complex and/or repetitive testing scenarios. In contrast, automated tools perform tests systematically and accurately, reducing the risk of missing potential security gaps. With the rise of agile and DevOps practices, AST practices allow continuous security checks without slowing down the development process, ensuring that security does not lag behind rapid development efforts. Automating security testing reduces the cost of manual labor and time spent on repetitive testing tasks. Over time, it could also lower the total cost of security management by identifying and mitigating vulnerabilities earlier in the development process, which is often less expensive than fixing issues postdeployment [160]. Despite all these benefits, AST is not without limitations. For example, it may not always detect sophisticated attacks or complex flaws that require human intuition. Although *[CH15]* discussed the lack of AST, the literature has not debated enough on how to technically verify and validate the effectiveness of implemented processes and tools used in AST, for example, static analysis, dynamic analysis, and container scanning. Therefore, additional research is needed to evaluate process effectiveness, improve tool accuracy, and develop hybrid approaches that effectively combine manual and automated testing.

While AST offers numerous advantages, the integration of AST tools into complex, multicloud environments remains a significant challenge and technical hurdle. This review highlights some key challenges that need to be addressed. For this reason, future research should explore advanced approaches and new methods, such as AI-driven testing, to enhance the precision and flexibility of AST practices [161]. Furthermore, there is a need for industry standards and frameworks to guide the selection and deployment of AST tools, confirming their effectiveness in diverse organizational contexts.

5.3 | Security Awareness and Training

Security awareness and training are critical for the organization's overall cybersecurity strategy, as they directly address one of the most significant vulnerabilities and major risks in any security system, which is human errors. Many security violations and malicious breaches result from human mistakes, such as using weak passwords, mishandling sensitive data, or falling victim to phishing attacks [162]. For this reason, security awareness and educating employees to recognize malicious attacks and common threats are essential for reducing the risk of cyberattacks by empowering employees with the knowledge and skills needed to recognize, avoid, and respond to threats. By fostering a culture of security, ensuring regulatory compliance, and staying ahead of evolving cyber threats, organizations can protect themselves against costly breaches and safeguard their digital systems and organizational assets

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

as well [49]. As suggested in *[SLN03]*, organizations are advised to invest in training their employees and follow a proactive, cost-effective strategy that strengthens the human element within the organizational cybersecurity defenses. This is a thorough approach to addressing the challenges related to people and employees like *[CH06, CH07, CH08, CH09]* and improving staff skills and performance. As a result, research should further investigate the positive impacts of security awareness and training programs on development and operation teams in DevSecOps environments.

On the other side, efficient SETA methods must be carefully assessed and validated due to their long-term impacts on security culture within organizations, and because current SETA methods may not adequately address evolving threats [163], requiring continuous research to ensure they remain effective. Additional research is required to create more customized SETA programs that accommodate cultural, educational, and demographic diversity, making training more effective across all user groups. Besides, research is needed to explore how people retain and act on security knowledge, the psychological aspects of training, and how to design engaging and retention-boosting programs. In reference to addressing the challenge of incorrect geographic distribution for DevSecOps teams *[CH16]*, organizations should map practical skills and resources in parallel with balancing collaboration, communication and operational efficiency in multiple locations. They are compelled to analyze, assess, and optimize team distribution efficiently to ensure key security skills are not isolated.

5.4 | Security Orchestration and Automation

This literature review underlines the growing importance of orchestration and automation in improving the effectiveness of security controls. As organizations grow and face increasingly complex IT infrastructures, including hybrid and multicloud environments, the manual management of security controls is no longer practical nor feasible [164]. Orchestration refers to the coordinated management of multiple security tools and processes, while automation involves the execution of tasks without human intervention. Together, these approaches streamline the implementation, monitoring, and enforcement of security controls [165].

Security automation and orchestration are essential components of modern cybersecurity strategies due to their ability to improve efficiency, reduce human error, and speed up threat response [166]. Automation helps reduce the time it takes to respond to cyber threats by automatically detecting, analyzing, and responding to incidents [167]. On the other hand, orchestration involves integrating and incorporating crucial security tools and systematic processes, allowing for automated and smooth workflows. Both automation and orchestration ensure that security incidents are handled consistently and quickly on different platforms [168]. Developers, similar to security analysts, need to genuinely understand the mechanisms of orchestration and automation. They should be aware of obstacles and difficulties that might be experienced during the conduct of automated security practices in DevSecOps pipelines. Such obstacles may lead to low adoption of DevSecOps practices [38]. For this reason, further research is needed to examine the orchestration and automation procedures in DevSecOps interfaces. Moreover, extra investigations are required to validate and evaluate the efficiency of automation processes conducted in IRM for the sake of responding against security attacks and improving resolution times, as pointed out in challenges *[CH15, CH18]* and solutions *[SLN05, SLN09, SLN12, SLN18]*.

In contrast, several studies highlighted the difficulty of integrating disparate security tools and legacy systems into a unified platform. For example, improperly configured automation workflows can inadvertently introduce risks, such as overprivileged access or misconfigured controls [169]. For this reason, future research should focus on developing standardized frameworks and best practices for secure implementation of orchestration and automation in diverse environments [170]. Moreover, ML and AI applications could contribute to further enhancing the adaptability and intelligence of automated workflows [171, 172].

5.5 | Security Metrics and Key Performance Indicators (KPIs)

Metrics and KPIs provide a quantifiable way to assess how well security controls are functioning. They help to identify whether implemented controls, like firewalls, intrusion detection systems, and encryption methods, are effective in preventing attacks and/or mitigating cyber threats [173]. Moreover, they highlight the areas of risks and vulnerabilities, such as weak password policies and unpatched systems, with the intention of enabling proactive mitigation [174]. Security metrics also allow organizations to align security efforts with business objectives by demonstrating how security controls support business continuity, legal compliance, and customer trust. In contrast, KPIs provide evidence of the return on investment (ROI) for security tools and initiatives, helping to justify budgets and resources allocated in executing security practices [175]. As a result, suggesting security metrics and KPIs in solution *[SLN05]* addressing challenge *[CH18]*, researching security metrics and KPIs is highly recommended to ensure that the chosen metrics are meaningful, actionable, and aligned with both security and business goals. By conducting thorough research in several areas like security metrics frameworks and metric types, organizations can develop robust and meaningful security metrics that align with their goals and continuously improve their cybersecurity posture.

5.6 | Threat Intelligence Integration

Threat intelligence integration is crucial for enhancing security defense mechanisms. It involves incorporating real-time and contextualized information about cyber threats (such as malware, threat actors, and attack techniques) into organizational security infrastructures. This preemptive approach helps organizations stay ahead of evolving threats [176]. Several challenges relating to security controls in threat integration intelligence are reported in this SLR *[CH03, CH11, CH12]*. However, the documentation limitation concerning this area makes the challenges, even more tricky and problematic

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

[CH01]. We have discovered that the main cause for such obstacles is development and use of traditional and old-fashioned security tools for security practitioners only, making it a bit burdensome for developers to learn or even utilize them. As a result, software practitioners find it quite difficult to implement DevSecOps practices within a large range of complex and heterogeneous components *[CH01, CH17, CH05, CH02]*, followed by extra activities within the development process *[SLN02, SLN06, SLN09, SLN11]*. This is considered a serious impediment to DevSecOps adoption, as developers must carry out the burden of security-related tasks. To conclude, this is a good opportunity to explore strategies and well-structured frameworks for integrating threat intelligence, including developers' insights about DevSecOps practices along with their feedback on hazard mitigation plans and IRM.

5.7 | Secure Data Containers

Secure data containers play a crucial role in protecting sensitive data, especially in today's digital landscape where data are constantly in transit and stored across various environments like clouds and mobile devices [177]. They allow organizations to isolate, encrypt, and protect organizational data across a range of networks and environments, ensuring compliance, reducing risks, and safeguarding against evolving threats and cyberattacks. In addition, these containers help ensure the CIA triad (confidentiality, integrity, and availability) of organizational data by providing secure and isolated environments to store and process sensitive information [178]. Some solutions *[SLN12, SLN17]* suggest using data containers to overcome scalability and reliability issues related to managing security controls in DevSecOps pipelines [179]. Developers and software practitioners still need to learn and practice the ideal approaches to appropriately implement data containers to address issues like *[CH18]*. Hence, research must extensively investigate the obstacles and challenges of using data containers to solve security problems and improve runtime applications.

5.8 | Continuous Feedback

Continuous feedback in software engineering is essential for maintaining high-quality code, enhancing collaboration, and enabling iterative improvements throughout the SDLC. In the fast-paced development process, regular feedback fosters a culture of agility, adaptability, and continuous learning [180]. For this reason, many studies reported that continuous feedback should be thoroughly considered and automated as much as possible at early stages within the SDLC [181], as pointed out in solution *[SLN03]*. Continuous feedback is proposed to instantly collect and understand user feedback *[SLN09]*. However, extra investigation is needed on how to obtain user feedback during the manual conduct of security practices to fit into DevSecOps pipelines. Regarding challenges *[CH06, CH11, CH12]*, the sourced papers did not fully inspect the optimization of feedback mechanisms. Therefore, researchers are advised to explore how to optimize feedback mechanisms for faster vulnerability remediation, analyze the effectiveness of feedback loops in DevSecOps particularly, and improve the overall security generally.

5.9 | Consensus on Shift-Left Security

Shift-left security is a core practice within DevSecOps that emphasizes incorporating and integrating security controls early in the SDLC. Security must be handled from the initial design phases rather than waiting until deployment or after the code is complete [182]. The term "shift-left" comes from the idea of moving security "to the left" in the development timeline, which traditionally placed security controls toward the end of the process [183]. In DevSecOps environments, security becomes a shared responsibility among all members working in the development, operations, and security teams, aiming to ensure that all vulnerabilities are spotted, identified, and addressed as early as possible [84]. As mentioned in challenge *[CH06]*, there is still no complete consensus about the best practices of DevSecOps due to the confusion and unclear information published in current literature about DevSecOps environments. Consequently, it is of utmost importance for researchers to illustrate the main concepts of DevSecOps along with "security-shift-left" best practices, with the aim of resolving possible conflicts among stakeholders and head managers concerning the implementation of security controls within DevSecOps pipelines.

5.10 | Need for Sociotechnical Studies

Sociotechnical studies is a collaborative and multidisciplinary field that examines the interaction between technology, society, and human behavior. They explore how technology influences social systems and how social and cultural factors, in turn, shape the development and use of technology [184]. In these studies, the learning is ultimately based on how the technical systems (e.g., machines, software applications, and infrastructures) and social systems (e.g., people and organizations) are intensely interconnected, and both must be considered together to design effective and sustainable solutions [185]. DevSecOps paradigm is widely recognized as a cultural transformation for organizations concerning the cybersecurity field [72, 186]. As pointed out by people-related challenges *[CH06, CH09, CH10]*, the lack of security skills can result in challenges and obstacles relating to agility, velocity, and integration. As a result, it is a good opportunity to conduct sociotechnical studies to foster security-conscious cultures and collaboration among development, security, and operations teams who work on DevSecOps practices.

5.11 | Security Testing in Multicloud Environments

Security testing in multicloud landscapes is essential due to the complexity, diversity, and dynamic nature of such architectures [187]. As organizations increasingly adopt multicloud networks and start working with external providers (e.g., AWS, Azure, and Google Cloud), ensuring the security of data and applications has become more challenging [188]. Concerning challenge *CH03*, obstacles may arise during the automation of security practices within multicloud environments [189]. Hence, researchers should pay more attention to demystifying the challenges related to building security controls during the execution of DevSecOps in multicloud interfaces. Advanced solutions must be proposed to support and maintain consistent security

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

controls across multicloud environments, as referred to in solution *[SLN02]*.

5.12 | Incident Response Management (IRM) in DevSecOps

IRM is indispensable in cybersecurity and business continuity because it ensures a well-organized and effective approach to detecting, mitigating, and recovering from several security incidents like cyberattacks, data breaches, and/or system failures [190]. Effective and practical IR plans would certainly narrow the impacts and potential damage caused by such incidents, reduce recovery time, and ensure compliance with security standards and regulatory requirements [191, 192]. Referring to the practices mentioned in *[SLN12, SLN18]*, there is still a need to examine IR processes to identify gaps in detecting, responding to, and recovering from malicious attacks during the implementation of security controls in DevSecOps. Researchers can investigate how IR teams can improve communication skills and IR capabilities from different perspectives (individuals, teams, and organizations), especially in high-stress situations. They can explore and inspect the optimal methods for isolating affected systems while minimizing the impact on operational transactions.

6 | Limitations and Threats to Validity

Like other surveys and systematic reviews, the findings could be impacted by potential threats combined with poor aggregation of primary studies [88, 89]. Relevant papers or suitable studies might have been missed. However, it is hard to guarantee that all relevant papers were captured in this SLR. We have thoroughly examined the risks to validity with the aim of addressing potential internal/external risks and enhancing both the credibility and reliability of the findings, thereby providing more accurate and meaningful insights to mitigate threats as much as possible. Due to limitations in research databases, narrow digital libraries, and search methodologies [99], we took multiple precautions to mitigate the risk of incomplete or biased literature coverage. We developed a thorough search strategy to include multiple libraries and source databases using a combination of wide-ranging synonyms, keywords, and related terms. Despite this, we acknowledge that one limitation of this literature review could be the potential incompleteness of the search string. While we aimed to use comprehensive and well-defined search terms, it is possible that certain relevant studies and papers were not captured due to the exclusion of alternative terms, keywords, or specific synonyms that might have been used by other researchers and missed out in this SLR. Further, we were keen to record every step and document the search process to ensure the replicability and transparency of data.

Bias resulting from picking up relevant papers by implementing inclusion/exclusion criteria normally differs due to subjective judgments and may affect literature reviews [193]. To reduce this effect, a study approach (protocol) was developed first to identify the RQs, search procedures, and inclusion/exclusion criteria [99]. The main objective of this approach was to increase the consistency in data collection and extraction procedures [194]. All stages (e.g., study selection and data extraction) of this SLR were shared among all authors to review and provide their feedback. The first author mainly conducted the data extraction process using data extraction spreadsheets. All related spreadsheets and work statements were shared with other authors to access and verify whenever needed. Regular meetings were held between authors to evaluate the progress and assess the outcome of each stage in this SLR. They confirmed and cross-checked the completion of paper selection, data extraction, and data analysis processes [193]. On the other side, we developed standardized spreadsheets for data extraction to handle inaccurate and/or inconsistent data collection and conducted multiple meetings to review and resolve data discrepancies. We also carried out pilot testing and ran a number of searches on prevalent databases and selected sources for the final primary papers. Further, the search string was improved in a set of iterations to capture relevant papers. Subsequently, the snowballing technique was carried out to capture more papers and explore the primary lists of other secondary studies reported in Table 2. Undertaking those activities initially resulted in duplicates and redundant articles, however, they noticeably reduced the chance of missing out on any relevant papers [100, 194]. Nevertheless, there is a risk of bias caused by not performing forward snowballing and neglecting those papers that could be identified through citation tracking. The paper selection might be limited to those prevalent or easily accessed papers.

To maintain a high standard of quality and integrity for our review, we included only peer-reviewed articles to ensure that the findings were based rigorously on research and evaluated by subject matter experts. In contrast, gray literature often lacks this formal evaluation process, leading to inconsistencies in reliability and credibility. Also, gray literature may not be accessible or uniformly indexed in some instances, making it difficult to include in SLRs. The scope of this review emphasized theoretical and empirical studies to align with our research objectives, which are less addressed in gray literature. To support replicability and transparency toward achieving the study goals, we included only the sources that provide detailed methodologies on security controls and DevSecOps practices. On the other side, gray literature often includes information and insights not published in traditional papers and academic journals like white papers, conference proceedings, and government publications. Many sources of gray literature usually come from security practitioners and industry professionals, making the acquired information more valuable and aligned with practical applications. Given these advantages of gray literature, including its ability to provide comprehensive coverage, real-world relevance, and a reduction in publication bias, we recognize its significant value in ensuring inclusive insights and expanding the scope of our research. We aim to conduct a broader future review and include gray literature, addressing new knowledge gaps and capturing additional hands-on practices and emerging trends.

7 | Conclusion

In this study, we conducted an SLR on 45 primary studies, which led to the following findings.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

- This SLR captures 19 security control–related security challenges and 18 solutions to address those challenges. Researchers are recommended to thoroughly consider other challenges and/or solutions within well-defined themes and proper categorization. Moreover, developers should conduct contingency plans and flexible workarounds to take appropriate actions to reduce the impacts of security issues when integrating security controls in DevSecOps.
- The focus area devised from this SLR is supporting security controls in DevSecOps. Current methods have a lot of deficiencies that would certainly hinder the integration of security controls/measures within CI/CD pipelines. Therefore, research for key studies on new methods to promote speedy deployments and rapid integration for security controls within DevSecOps pipelines is a current requirement.
- It is difficult to fully incorporate security controls in DevSecOps using outdated techniques. These techniques hinder the adoption of DevSecOps as they are unable to automate security practices. This is a tricky issue, considering automation is a major pillar in DevSecOps. Hence, further research is required to support automation in continuous security practices within CI/CD pipelines. Security practitioners need to know how to mitigate issues of manual security testing and reduce negative impacts when adding automation to security practices particularly. Thereby, key studies are needed to explore suitable trade-offs and maintain the objective symmetry of both agility and DevSecOps.
- This SLR introduces some staff-related challenges like conflict management and communication issues among individuals. Although collaboration and cultural factors are critical for DevSecOps adoption, they are not investigated enough in the current literature. This fact brings up a strong recommendation to conduct up-to-the-minute research exploring sociotechnical studies concerning staff working in DevSecOps. New research should explore more key studies where security practitioners faced social issues due to people problems (e.g., grudges, negative conversations, misconceptions, or stereotypes between departments). Consequently, security analysts will get a better opportunity to understand how such issues were previously handled and mitigated. Eventually, they will greatly benefit from such research to reduce multidisciplinary problems and obtain a list of valuable lessons learned from past experiences.
- DevSecOps is developed based on shifting security to the left and integrating security controls at the early stages of SDLC to ensure code is securely written/tested and reduce resolution time for cyberattacks. This paper highly recommends shifting security to the left and executing continuous scanning for security practices to sustain a solid security posture throughout the deployment cycle.

In conclusion, the suggested solutions provide a practical foundation for implementing and integrating robust security controls within the continuous practices of DevSecOps pipelines. By addressing the key challenges such as automation, scalability, and real-time responsiveness, those approaches and solutions would genuinely enable security practitioners to enhance overall security posture without compromising the agility and efficiency that CD demands. The integration of these security controls fosters a solid and stable culture of proactive security, aligning development, operations, and security teams toward a unified goal of delivering secure and resilient software. This alignment not only mitigates risks effectively but also empowers these teams to collaborate and embed security controls seamlessly into their workflows, ultimately supporting the realization of secure, high-performing systems in dynamic environments.

Data Availability Statement

The data that support the findings of this study are available on request from the corresponding author. The data are not publicly available due to privacy or ethical restrictions.

Endnotes

[1] Transport layer security and secure sockets layer.

[2] General Data Protection Regulation.

[3] International Organization for Standardization and the International Electrotechnical Commission.

[4] https://checkmarx.com/cxsast-source-code-scanning/.

References

1. F. Lombardi and A. Fanton, "From DevOps to DevSecOps Is Not Enough. CyberDevOps: An Extreme Shifting-Left Architecture to Bring Cybersecurity Within Software Security Lifecycle Pipeline," *Software Quality Journal* 31, no. 2 (2023): 619–654.

2. B. S. Farroha and D. L. Farroha, "A Framework for Managing Mission Needs, Compliance, and Trust in the DevOps Environment," in *2014 IEEE Military Communications Conference* (IEEE, 2014): 288–293.

3. L. J. Zhang, J. Zhang, and H. Cai, "End-to-End Services Delivery Platform and Methodology," in *Services Computing* (Springer, 2007): 310–329.

4. S. Radack, *The System Development Life Cycle (SDLC).* tech. rep. (National Institute of Standards and Technology, 2009).

5. S. Alajrami, A. Romanovsky, and B. Gallina, "Software Development in the Post-PC Era: Towards Software Development as a Service," in *Product-Focused Software Process Improvement: 17th International Conference, PROFES 2016* (Springer, 2016): 662–671.

6. E. C. Burkard, "Usability Testing Within a DevSecOps Environment," in *2020 Integrated Communications Navigation and Surveillance Conference (ICNS)* (IEEE, 2020).

7. H. Myrbakken and R. Colomo-Palacios, *DevSecOps: A Multivocal Literature Review.* tech. rep. (SpringerLink, 2017).

8. V. Mohan and L. B. Othmane, "DevSecOps: Is It a Marketing Buzzword?—Mapping Research on Security in DevOps," in *2016 11th International Conference on Availability, Reliability and Security (ARES)* (IEEE, 2016), 542–547.

9. J. C. Akujobi, *A Model for Measuring Improvement of Security in Continuous Integration Pipelines: Metrics & Four-Axis Maturity Driven DevSecOps (MFAM).* tech. rep. (University of Twente, 2021).

10. L. Prates, J. Faustino, M. Silva, and R. Pereira, "DevSecOps Metrics," in *Information Systems: Research, Development, Applications, Education: 12th SIGSAND/PLAIS EuroSymposium 2019* (Springer, 2019): 77–90.

11. A. Ullah and R. Lai, "Modeling Business Goal for Business/IT Alignment Using Requirements Engineering," *Journal of Computer Engineer Systems* 51, no. 3 (2015): 21–28.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

12. G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook: How to Create World-Classy Agility, Reliability and Security in IT Organisations* (IT Revolution, 2021).

13. A. Sharma and P. K. Misra, "Aspects of Enhancing Security in Software Development Life Cycle," *Advances in Computational Sciences and Technology* 10, no. 2 (2017): 203–210.

14. R. A. Khan, M. A. Akbar, S. Rafi, A. O. Almagrabi, and M. Alzahrani, "Evaluation of Requirement Engineering Best Practices for Secure Software Development in GSD: An ISM Analysis," *Journal of Software: Evolution and Process* 36, no. 5 (2024): e2594.

15. J. Zhou, "DevSecOps: Integrating Security in DevOps for Financial Applications," in *Future And Fintech, The: Abcdi And Beyond* (World Scientific, 2022): 423–450.

16. Strategic IT Governance and Performance Frameworks in Large Organizations, *From Information Governance to IT Governance: An Overview of Issues and Frameworks for Large Organizations* (IGI Global, 2019).

17. D. Verslegers, "Challenges and Opportunities for Security Assurance in DevOps," in *Strategic Approaches to Digital Platform Security Assurance* (IGI Global, 2021).

18. Y. Kurii and I. Opirskyy, "Analysis and Comparison of the NIST SP 800-53 and ISO/IEC 27001:2013," *NIST Special Publication* 800, no. 53 (2022): 21–32.

19. H. Susanto and M. N. Almunawar, *Information Security Management Systems: A Novel Framework and Software as a Tool for Compliance With Information Security Standard* (Apple Academic Press., 2018).

20. A. Bicaku, M. Tauber, and J. Delsing, "Security Standard Compliance and Continuous Verification for Industrial Internet of Things," *International Journal of Distributed Sensor Networks* 16, no. 6 (2020): 1550147720922731.

21. M. Alicea and I. Alsmadi, "Misconfiguration in Firewalls and Network Access Controls: Literature Review," *Future Internet* 13, no. 11 (2021): 283.

22. J. A. Ruíz-Tapia, S. Ruíz-Valdés, I. R. Cruz-Solís, and F. H. Alcántara-Cruz, "Administrative Audit Tool as Support for the ISO 27001 Standard Towards Managing the Quality of Information in SMSEs, 2021," *Journal of Business & SMEs* 8, no. 21 (2022): 11–23.

23. M. Butavicius, K. Parsons, M. Lillie, A. McCormac, M. Pattinson, and D. Calic, "When Believing in Technology Leads to Poor Cyber Security: Development of a Trust in Technical Controls Scale," *Computers & Security* 98 (2020): 102020.

24. R. Makri, P. Karaivazoglou, A. Kyritsis, et al., "Modern Innovative Detectors of Physical Threats for Critical Infrastructures," in *Cyber-Physical Threat Intelligence for Critical Infrastructures Security*, vol. 397 (Now Publishers, 2020).

25. L. Collins, *Computer and Information Security Handbook (Second Edition)*, 2nd ed. (Morgan Kaufmann, 2013), 1015–1021.

26. B. Alluhaybi, M. S. Alrahhal, A. Alzhrani, and V. Thayananthan, "A Survey: Agent-Based Software Technology Under the Eyes of Cyber Security, Security Controls, Attacks and Challenges," *International Journal of Advanced Computer Science and Applications (IJACSA)* 10, no. 8 (2019): 211–230.

27. S. Maksuti, A. Bicaku, M. Zsilak, et al., "Automated and Secure Onboarding for System of Systems," *IEEE Access* 9 (2021): 111095–111113.

28. K. Kaur, I. Gupta, A. K. Singh, et al., "A Comparative Evaluation of Data Leakage/Loss Prevention Systems (DLPS)," in *Proceedings of the 4th International Conference on Computer Science & Information Technology (CS & IT-CSCP)* (AIRCC Publishing Corporation, 2017), 87–95.

29. S. O. Olabanji, O. O. Olaniyi, C. S. Adigwe, O. J. Okunleye, and T. O. Oladoyinbo, "AI for Identity and Access Management (IAM) in the Cloud: Exploring the Potential of Artificial Intelligence to Improve User Authentication, Authorization, and Access Control Within Cloud-Based Systems," *Asian Journal of Research in Computer Science* 17, no. 3 (2024): 38–56.

30. A. Arfeen, S. Ahmed, M. A. Khan, and J. S. F. A., "Endpoint Detection & Response: A Malware Identification Solution," in *2021 International Conference on Cyber Warfare and Security (ICCWS)* (IEEE, 2021), 1–8.

31. F. G. Siji and O. P. Uche, "An Improved Model for Comparing Different Endpoint Detection and Response Tools for Mitigating Insider Threat," *Indian Journal of Engineering* 20, no. 53 (2023): 1–13.

32. M. L. Hale and R. F. Gamble, "Semantic Hierarchies for Extracting, Modeling, and Connecting Compliance Requirements in Information Security Control Standards," *Requirements Engineering* 24 (2019): 365–402.

33. M. Parker, B. Partridge, E. Svetcov, and A. J. Taylor, "Managing Third-Party Risk: Framework Details for Risk Management in Medical Technology," in *Advanced Health Technology* (Productivity Press, 2023), 213–252.

34. K. Ryanto and V. Tundjungsari, "Standardization of Information Security Management in the Banking Sector using the ISO 27001: 2022 Framework," *Journal La Multiapp* 5, no. 4 (2024): 361–379.

35. C. Colabuono, D. Wiemer, M. V. Marabello, et al., "Approach to Sector-Specific Cybersecurity Schemes: Key Elements and Security Problem Definition," in *International Conference on Multimedia Communications, Services and Security* (Springer, 2022), 104–117.

36. M. Malatji, "Management of Enterprise Cyber Security: A Review of ISO/IEC 27001: 2022," in *2023 International Conference on Cyber Management and Engineering (CyMaEn)* (IEEE, 2023), 117–122.

37. S. Watkins, *ISO/IEC 27001: 2022: An Introduction to Information Security and the ISMS Standard* (*Torrossa*, 2022).

38. V. Mohan, L. Othmane, and A. Kres, "BP: Security Concerns and Best Practices for Automation of Software Deployment Processes: An Industrial Case Study," in *2018 IEEE Cybersecurity Development (SecDev)* (IEEE, 2018).

39. S. V. Deshmukh, D. S. Ahire, N. N. Chavan, N. D. Bharambe, and A. R. Jain, *Implementing DevSecOps Pipeline for an Enterprise Organization* (*Springer*, 2021).

40. M. Fowler, *Refactoring: Improving the Design of Existing Code* (Addison-Wesley Professional, 2018).

41. M. Kaya, S. Conley, Z. S. Othman, and A. Varol, "Effective Software Refactoring Process," in *2018 6th International Symposium on Digital Forensic and Security (ISDFS)* (IEEE, 2018), 1–6.

42. R. Saint-Germain, "Information Security Management Best Practice Based on ISO/IEC 17799," *Information Management Journal-Prairie Village* 39, no. 4 (2005): 60–66.

43. K. Cormican, "Integrated Enterprise Risk Management: From Process to Best Practice," *Modern Economy* 5, no. 4 (2014): 44995.

44. M. Trnka, A. S. Abdelfattah, A. Shrestha, M. Coffey, and T. Cerny, "Systematic Review of Authentication and Authorization Advancements for the Internet of Things," *Sensors* 22, no. 4 (2022): 1361.

45. R. S. Kale, J. Hase, S. Deshmukh, S. N. Ajani, P. K. Agrawal, and C. S. Khandelwal, "Ensuring Data Confidentiality and Integrity in Edge Computing Environments: A Security and Privacy Perspective," *Journal of Discrete Mathematical Sciences and Cryptography* 27 (2024): 421–430.

46. P. Sirohi, A. Agarwal, and S. Tyagi, "A Comprehensive Study on Security Attacks on SSL/TLS Protocol," in *2016 2nd International Conference on Next Generation Computing Technologies (NGCT)* (IEEE, 2016), 893–898.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

47. G. González-Granadillo, S. González-Zarzosa, and R. Diaz, "Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures," *Sensors* 21, no. 14 (2021): 4759.

48. P. Sharma, "Enforcing Network Security Policies in the Context of the NIST Cybersecurity Framework and Its Legal Implications," *Network Security* 2024, no. 7 (2024): 22–27.

49. T. Stefaniuk, "Training in Shaping Employee Information Security Awareness," *Entrepreneurship and Sustainability Issues* 7, no. 3 (2020): 1832–1846.

50. U. Hamza, N. L. Abdullah, and S. M. Syed-Mohamad, "DevOps Adoption Guidelines, Challenges and Benefits: A Systematic Literature Review," *Journal of Advanced Research in Applied Sciences and Engineering Technology* 61, no. 4 (2024): 115–136.

51. J. Faustino, D. Adriano, R. Amaro, R. Pereira, and M. Mira da Silva, "DevOps Benefits: A Systematic Literature Review," *Software: Practice and Experience* 52, no. 9 (2022): 1905–1926.

52. F. El Aouni, K. Moumane, A. Idri, M. Najib, and S. U. Jan, "A Systematic Literature Review on Agile, Cloud, and DevOps Integration: Challenges, Benefits," *Information and Software Technology* 177 (2024): 107569.

53. IEEE Standards Committee, *IEEE Standard for DevOps: Building Reliable and Secure Systems Including Application Build, Package, and Deployment: IEEE Standard 2675-2021* (IEEE, 2021).

54. K. Neharika and R. G. Lennon, "Investigations into Secure IaC Practices," in *Proceedings of Seventh International Congress on Information and Communication Technology: ICICT 2022*, vol. 2 (Springer, 2022), 289–303.

55. C. Blundo, S. Cimato, and L. Siniscalchi, "Managing Constraints in Role Based Access Control," *IEEE Access* 8 (2020): 140497–140511.

56. S. M. Mohammad, "Continuous Integration and Automation," *International Journal of Creative Research Thoughts (IJCRT)* 4, no. 3 (2016): 2320–2882.

57. L. E. Lwakatare, P. Kuvaja, and M. Oivo, "Relationship of DevOps to Agile, Lean and Continuous Deployment: A Multivocal Literature Review Study," in *Product-Focused Software Process Improvement: 17th International Conference, PROFES 2016* (Springer, 2016), 399–415.

58. Y. Fan, X. Wang, T. Funk, et al., "A Critical Review for Real-Time Continuous Soil Monitoring: Advantages, Challenges, and Perspectives," *Environmental Science & Technology* 56, no. 19 (2022): 13546–13564.

59. X. Zhou, R. Mao, H. Zhang, et al., "Revisit Security in the Era of DevOps: An Evidence-Based Inquiry Into DevSecOps Industry," *IET Software* 17, no. 4 (2023): 435–454.

60. A. Sen, "DevOps, DevSecOps, AIOPS-Paradigms to IT Operations," in *Evolving Technologies for Computing, Communication and Smart World* (Springer, 2021): 211–221.

61. R. Mao, H. Zhang, Q. Dai, et al., "Preliminary Findings About DevSecOps From Grey Literature," in *2020 IEEE 20th International Conference on Software Quality, Reliability and Security (QRS)* (IEEE, 2020), 450–457.

62. H. Adkins, B. Beyer, P. Blankinship, P. Lewandowski, A. Oprea, and A. Stubblefield, *Building Secure & Reliable Systems-Best Practices for Designing, Implementing and Maintaining Systems* (O'Reilly Media Inc., 2020).

63. L. Wehenkel and M. Pavella, "Decision Tree Approach to Power Systems Security Assessment," *International Journal of Electrical Power & Energy Systems* 15, no. 1 (1993): 13–36, https://doi.org/10.1016/0142-0615(93)90014-E.

64. M. Zaydi and B. Nassereddine, "DevSecOps Practices for an Agile and Secure IT Service Management," *Journal of Management Information and Decision Sciences* 23, no. 2 (2020): 134–149.

65. M. Hadi and H. Bahi, *Making the Shift From DevOps to DevSecOps at Distribution Technologies* (TAL TECH GMBH, 2019).

66. Y. Bobbert and M. Chtepen, *Research Findings in the Domain of CI/CD and DevOps on Security Compliance* (*IGI Publisher of Timely Knowledge*, 2021).

67. H. Fujita, A. Selamat, and S. Omatu, *Knowledge Innovation Through Intellignet Software Methodologies, Tools and Techniques* (IOS Press, 2020).

68. S. Dupont, G. Ginis, M. Malacario, et al., "Incremental Common Criteria Certification Processes Using DevSecOps Practices," in *2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)* (IEEE, 2021).

69. E. Sermpezis, "Integration of Security in the DevOps Methodology," in *2024 15th International Conference on Information, Intelligence, Systems & Applications (IISA)* (IEEE, 2024).

70. C. A. Cois, J. Yankel, and A. Connell, "Modern DevOps: Optimizing Software Development Through Effective System Interactions," in *2014 IEEE International Professional Communication Conference (IPCC)* (IEEE, 2014), 1–7.

71. D. W. Hallock, *DevSecOps in Practice* (CrossTalk, 2022).

72. R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and Solutions When Adopting DevSecOps: A Systematic Review," *Information and Software Technology* 141 (2022): 106700.

73. Y. Zhang, B. Vasilescu, H. Wang, and V. Filkov, "One Size Does Not Fit All: An Empirical Study of Containerized Continuous Deployment Workflows," in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Association for Computing Machinery, 2018), 295–306.

74. T. Detofeno, A. Malucelli, and S. Reinehr, "PriorTD: A Method for Prioritization Technical Debt," in *Proceedings of the XXXVI Brazilian Symposium on Software Engineering* (Association for Computing Machinery, 2022), 230–240.

75. G. Park, "The Changing Wind of Data Privacy Law: A Comparative Study of the European Union's General Data Protection Regulation and the 2018 California Consumer Privacy Act," *UC Irvine Law Review* 10 (2019): 1455–1490.

76. N. Lamponen, *Implementation of Secure Workflow for DevOps From Best Practices Viewpoint* (*LUT University*, 2021).

77. G. G. Claps, R. B. Svensson, and A. Aurum, "On the Journey to Continuous Deployment: Technical and Social Challenges Along the Way," *Information and Software Technology* 57 (2015): 21–31.

78. J. Humble and J. Molesky, "Why Enterprises Must Adopt DevOps to Enable Continuous Delivery," *Cutter IT Journal* 24 (2011): 6.

79. H. Dhaduk, *DevOps Lifecycle: 7 Phases Explained in Detail With Examples* (*SIMFORM*, 2022).

80. E. Ntentos, *Supporting Architecture Evolution in Microservice-Based Systems and Infrastructure-as-Code Based Deployments* (*Wien University*, 2023).

81. A. Wiedemann, M. Wiesche, H. Gewald, and H. Krcmar, "Understanding How DevOps Aligns Development and Operations: A Tripartite Model of Intra-IT Alignment," *European Journal of Information Systems* 29, no. 5 (2020): 458–473.

82. B. Fitzgerald and K. J. Stol, "Continuous Software Engineering: A Roadmap and Agenda," *Journal of Systems and Software* 123 (2017): 176–189.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

83. M. Hvardand and C. P. Ricardo, "DevSecOps: A Multivocal Literature Review," in *Software Process Improvement and Capability Determination. SPICE 2017* (Springer, 2017), 17–29.

84. M. Sánchez-Gordón and R. Colomo-Palacios, "Security as Culture: A Systematic Literature Review of DevSecOps," in *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops* (Association for Computing Machinery, 2020), 266–269.

85. S. Rafi, W. Yu, M. A. Akbar, A. Alsanad, and A. Gumaei, "Multicriteria Based Decision Making of DevOps Data Quality Assessment Challenges Using Fuzzy TOPSIS," *IEEE Access* 8 (2020): 46958–46980.

86. S. Rafi, W. Yu, M. A. Akbar, A. Alsanad, and A. Gumaei, "Prioritization Based Taxonomy of DevOps Security Challenges Using PROMETHEE," *IEEE Access* 8 (2020): 105426–105446.

87. M. A. Akbar, K. Smolander, S. Mahmood, and A. Alsanad, "Toward Successful DevSecOps in Software Development Organizations: A Decision-Making Framework," *Information and Software Technology* 147 (2022): 106894.

88. X. Ramaj, M. Sánchez-Gordón, V. Gkioulos, S. Chockalingam, and R. Colomo-Palacios, "Holding on to Compliance While Adopting DevSecOps: An SLR," *Electronics* 11, no. 22 (2022): 3707.

89. M. Chiari, M. De Pascalis, and M. Pradella, "Static Analysis of Infrastructure As Code: A Survey," in *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)* (IEEE, 2022), 218–225.

90. F. M. Erich, C. Amrit, and M. Daneva, "A Qualitative Study of DevOps Usage in Practice," *Journal of Software: Evolution and Process* 29, no. 6 (2017): e1885.

91. B. A. Kitchenham, T. Dyba, and M. Jorgensen, "Evidence-Based Software Engineering," in *Proceedings. 26th International Conference on Software Engineering* (IEEE, 2004), 273–281.

92. W. Ding, P. Liang, A. Tang, and H. Van Vliet, "Knowledge-Based Approaches in Software Documentation: A Systematic Literature Review," *Information and Software Technology* 56, no. 6 (2014): 545–567.

93. B. A. Kitchenham, "Systematic Review in Software Engineering: Where We Are and Where We Should Be Going," in *Proceedings of the 2nd International Workshop on Evidential Assessment of Software Technologies* (Association for Computing Machinery, 2012), 1–2.

94. T. Bikis, "SAFe and DevSecOps in Governmental Organizations: A Case Study for Benefits and Challenges," (2022).

95. C. Colliander, *Challenges of DevSecOps*" (Master's thesis (University of Helsinki, Faculty of Science, 2022), http://hdl.handle.net/10138/342887.

96. R. Desai and T. Nisha, "Best Practices for Ensuring Security in DevOps: A Case Study Approach," *Journal of Physics: Conference Series* 1964, no. 4 (2021): 042045.

97. H. Zhang, M. A. Babar, and P. Tell, "Identifying Relevant Studies in Software Engineering," *Information and Software Technology* 53, no. 6 (2011): 625–637.

98. L. Chen, M. A. Babar, and H. Zhang, "Towards an Evidence-Based Understanding of Electronic Data Sources," in *14th International Conference on Evaluation and Assessment in Software Engineering (EASE)* (BCS Learning & Development Ltd., 2010), 1–4.

99. T. Dybå and T. Dingsøyr, "Empirical Studies of Agile Software Development: A Systematic Review," *Information and Software Technology* 50, no. 9-10 (2008): 833–859.

100. C. Wohlin, "Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering," in *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering* (Association for Computing Machinery, 2014), 1–10.

101. I. Koren, F. Rinker, K. Meixner, J. Matevska, and J. Walter, "Challenges and Opportunities of DevOps in Cyber-Physical Production Systems Engineering," in *2023 IEEE 6th International Conference on Industrial Cyber-Physical Systems (ICPS)* (IEEE, 2023), 1–6.

102. M. Senapathi, J. Buchan, and H. Osman, "DevOps Capabilities, Practices, and Challenges: Insights From a Case Study," in *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018* (Association for Computing Machinery, 2018), 57–67.

103. N. Beigi-Mohammadi, M. Litoiu, M. Emami-Taba, et al., "A DevOps Framework for Quality-Driven Self-Protection in Web Software Systems," in *Proceedings of the 28th Annual International Conference on Computer Science and Software Engineering* (IBM Corp., 2018), 270–274.

104. S. Rafi, W. Yu, and M. A. Akbar, "Towards a Hypothetical Framework to Secure DevOps Adoption: Grounded Theory Approach," in *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering* (Association for Computing Machinery, 2020), 457–462.

105. R. Filepp, C. Adam, M. Hernandez, M. Vukovic, N. Anerousis, and G. Q. Zhang, "Continuous Compliance: Experiences, Challenges, and Opportunities," in *2018 IEEE World Congress on Services (SERVICES)* (IEEE, 2018), 31–32.

106. R. Namal Rajapakse, M. Zahedi, and M. A. Babar, "An Empirical Analysis of Practitioners' Perspectives on Security Tool Integration into DevOps," preprint, arXiv, July 19, 2021, https://doi.org/10.48550/arXiv.2107.02096.

107. R. Kumar and R. Goyal, "Modeling Continuous Security: A Conceptual Model for Automated DevSecOps Using Open-Source Software Over Cloud (ADOC)," *Computers & Security* 97 (2020): 101967.

108. Z. Ahmed and S. C. Francis, "Integrating Security With DevSecOps: Techniques and Challenges," in *2019 International Conference on Digitization (ICD)* (IEEE, 2019), 178–182.

109. A. Qumer Gill, A. Loumish, I. Riyat, and S. Han, "DevOps for Information Management Systems," *VINE Journal of Information and Knowledge Management Systems* 48, no. 1 (2018): 122–139.

110. M. Efendi, T. Raharjo, and A. Suhanto, "DevSecOps Approach in Software Development Case Study: Public Company Logistic Agency," in *2021 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS)* (IEEE, 2021), 96–101.

111. J. R. Michener and A. T. Clager, "Mitigating an Oxymoron: Compliance in a DevOps Environments," in *2016 IEEE 40th Annual Computer Software and Applications Conference (COMPSAC)* (IEEE, 2016), 396–398.

112. G. Liu, B. Huang, Z. Liang, M. Qin, H. Zhou, and Z. Li, "Microservices: Architecture, Container, and Challenges," in *2020 IEEE 20th international conference on software quality, reliability and security companion (QRS-C)* (IEEE, 2020), 629–635.

113. B. Fitzgerald, K. J. Stol. Continuous Software Engineering and Beyond: Trends and Challenges. In: Proceedings of the 1st International Workshop on rapid continuous software engineering. (2014): 1–9.

114. V. Mohan, L. Othmane, and A. Kres, "BP: Security Concerns and Best Practices for Automation of Software Deployment Processes: An Industrial Case Study," in *2018 IEEE Cybersecurity Development (SecDev)* (IEEE, 2018), 21–28.

115. M. A. Aljohani and S. S. Alqahtani, "A Unified Framework for Automating Software Security Analysis in DevSecOps," in *2023 International Conference on Smart Computing and Application (ICSCA)* (IEEE, 2023), 1–6.

116. M. Voggenreiter and U. Schöpp, "Using a Semantic Knowledge Base to Improve the Management of Security Reports in Industrial DevOps Projects," in *Proceedings of the 44th International Conference*

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

on Software Engineering: Software Engineering in Practice (IEEE, 2022), 309–310.

117. J. Alonso, R. Piliszek, and M. Cankar, "Embracing IaC Through the DevSecOps Philosophy: Concepts, Challenges, and a Reference Framework," *IEEE Software* 40, no. 1 (2022): 56–62.

118. J. A. Morales, T. P. Scanlon, A. Volkmann, J. Yankel, and H. Yasar, "Security Impacts of Sub-Optimal DevSecOps Implementations in a Highly Regulated Environment," in *Proceedings of the 15th International Conference on Availability, Reliability and Security* (Association for Computing Machinery, 2020), 1–8.

119. S. Throner, H. Hütter, N. Sänger, et al., "An Advanced DevOps Environment for Microservice-Based Applications," in *2021 IEEE International Conference on Service-Oriented System Engineering (SOSE)* (IEEE, 2021), 134–143.

120. M. Shahin, M. A. Babar, M. Zahedi, and L. Zhu, "Beyond Continuous Delivery: An Empirical Investigation of Continuous Deployment Challenges," in *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)* (IEEE, 2017), 111–120.

121. H. Kang, M. Le, and S. Tao, "Container and Microservice Driven Design for Cloud Infrastructure DevOps," in *2016 IEEE International Conference on Cloud Engineering (IC2E)* (IEEE, 2016), 202–211.

122. C. Parnin, E. Helms, C. Atlee, et al., "The Top 10 Adages in Continuous Deployment," *IEEE Software* 34, no. 3 (2017): 86–95.

123. L. Chen, "Continuous Delivery: Huge Benefits, but Challenges Too," *IEEE Software* 32, no. 2 (2015): 50–54.

124. P. Stöckle, I. Pruteanu, B. Grobauer, and A. Pretschner, "Hardening With Scapolite: A DevOps-Based Approach for Improved Authoring and Testing of Security-Configuration Guides in Large-Scale Organizations," in *Proceedings of the Twelfth ACM Conference on Data and Application Security and Privacy* (Association for Computing Machinery, 2022), 137–142.

125. A. Saboor, M. F. Hassan, R. Akbar, et al., "Root-of-Trust for Continuous Integration and Continuous Deployment Pipeline in Cloud Computing," *Computers, Materials & Continua* 73, no. 2 (2022): 2223–2239.

126. T. Okubo and H. Kaiya, "Efficient Secure DevOps Using Process Mining and Attack Defense Trees," *Procedia Computer Science* 207 (2022): 446–455.

127. I. A. Tøndel and G. Brataas, "SecureScale: Exploring Synergies Between Security and Scalability in Software Development and Operation," in *Proceedings of the 2022 European Interdisciplinary Cybersecurity Conference* (Association for Computing Machinery, 2022), 36–41.

128. A. Dalvi, "Cloud Infrastructure Self Service Delivery System using Infrastructure as Code," in *2022 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)* (IEEE, 2022), 1–6.

129. T. Nisha and A. Khandebharad, "Migration From DevOps to DevSecOps: A Complete Migration Framework, Challenges, and Evaluation," *International Journal of Cloud Applications and Computing (IJCAC)* 12, no. 1 (2022): 1–15.

130. J. Cusick, A. Avritzer, A. Tse, and A. Janes, "Automated Dependability Assessment in DevOps Environments," in *2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)* (IEEE, 2022), 93–98.

131. A. Avritzer, "Challenges and Approaches for the Assessment of Micro-Service Architecture Deployment Alternatives in DevOps: A Tutorial Presented at ICSA 2020," in *2020 IEEE International Conference on Software Architecture Companion (ICSA-C)* (IEEE, 2020), 1–2.

132. S. Carturan and D. Goya, "Major Challenges of Systems-of-Systems With Cloud and DevOps—A Financial Experience Report," in *2019 IEEE/ACM 7th International Workshop on Software Engineering for Systems-of-Systems (SESoS) and 13th Workshop on Distributed Software Development, Software Ecosystems and Systems-of-Systems (WDES)* (IEEE, 2019), 10–17.

133. M. Staples, L. Zhu, and J. Grundy, "Continuous Validation for Data Analytics Systems," in *Proceedings of the 38th International Conference on Software Engineering Companion* (Association for Computing Machinery, 2016), 769–772.

134. L. Liu, D. Xie, Y. Cheng, and G. Li, "Architecture Scheme of DevOps for Cross Network and Multiple Environment Collaboration," in *Proceedings of the 5th International Conference on Computer Science and Application Engineering* (Association for Computing Machinery, 2021), 1–5.

135. M. Khanam, S. Rafi, and M. A. Akbar, "Understandings of Ethics in DevOps Teams: A Preliminary Investigation," in *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering* (Association for Computing Machinery, 2023), 394–397.

136. G. Casale, C. Chesta, P. Deussen, et al., "Current and Future Challenges of Software Engineering for Services and Applications," *Procedia Computer Science* 97 (2016): 34–42.

137. J. Díaz, J. E. Pérez, M. A. Lopez-Peña, G. A. Mena, and A. Yagüe, "Self-Service Cybersecurity Monitoring as Enabler for DevSecOps," *IEEE Access* 7 (2019): 100283–100295.

138. M. Alawneh and I. M. Abbadi, "Expanding DevSecOps Practices and Clarifying the Concepts Within Kubernetes Ecosystem," in *2022 Ninth International Conference on Software Defined Systems (SDS)* (IEEE, 2022), 1–7.

139. A. Sadovykh, G. Widforss, D. Truscan, et al., "VeriDevOps: Automated Protection and Prevention to Meet Security Requirements in DevOps," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)* (IEEE, 2021), 1330–1333.

140. A. Valani, "Rethinking Secure DevOps Threat Modeling: The Need for a Dual Velocity Approach," in *2018 IEEE Cybersecurity Development (SecDev)* (IEEE, 2018), 136.

141. V. Garousi, M. Felderer, and M. V. Mäntylä, "Guidelines for Including Grey Literature and Conducting Multivocal Literature Reviews in Software Engineering," *Information and Software Technology* 106 (2019): 101–121.

142. M. I. Alhojailan, "Thematic Analysis: A Critical Review ofits Process and Evaluation," in *WEI International European Academic Conference Proceedings* (Citeseer, 2012).

143. V. Braun and V. Clarke, "Using Thematic Analysis in Psychology," *Qualitative Research in Psychology* 3, no. 2 (2006): 77–101.

144. A. G. Tuckett, "Applying Thematic Analysis Theory to Practice: A Researcher's Experience," *Contemporary Nurse* 19, no. 1-2 (2005): 75–87.

145. R. Jabbari, N. Ali, K. Petersen, and B. Tanveer, "Towards a Benefits Dependency Network for DevOps Based on a Systematic Literature Review," *Journal of Software: Evolution and Process* 30, no. 11 (2018): e1957.

146. D. S. Cruzes and T. Dyba, "Recommended Steps for Thematic Synthesis in Software Engineering," in *2011 International Symposium on Empirical Software Engineering and Measurement* (IEEE, 2011), 275–284.

147. J. L. Campbell, C. Quincy, J. Osserman, and O. K. Pedersen, "Coding In-Depth Semistructured Interviews: Problems of Unitization and Intercoder Reliability and Agreement," *Sociological Methods & Research* 42, no. 3 (2013): 294–320.

148. V. Diamantopoulou, A. Tsohou, and M. Karyda, "From ISO/IEC 27002: 2013 Information Security Controls to Personal Data Protection Controls: Guidelines for GDPR Compliance," in *Computer Security: ESORICS 2019 International Workshops, CyberICPS, SECPRE, SPOSE, and ADIoT* (Springer, 2020), 238–257.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

149. K. Singh and P. Best, "Auditing During a Pandemic—Can Continuous Controls Monitoring (CCM) Address Challenges Facing Internal Audit Departments?," *Pacific Accounting Review* 35, no. 5 (2023): 727–745.

150. C. Barria, D. Cordero, L. Galeazzi, and A. Acuña, "Proposal of a Multi-Standard Model for Measuring Maturity Business Levels With Reference to Information Security Standards and Controls," in *Intelligent Methods in Computing, Communications and Control. ICCCC 2020* (Springer, 2021), 121–132.

151. A. Rajamäki, *Industrial Control Systems' Integrations to Operation Technology and Information Technology Security Operation Center* (*Jamaki*, 2021).

152. B. Gallina, T. Y. Olesen, E. Parajdi, and M. Aarup, "A Knowledge Management Strategy for Seamless Compliance With the Machinery Regulation," in *European Conference on Software Process Improvement* (Springer, 2023), 220–234.

153. A. J. Apeh, A. O. Hassan, O. O. Oyewole, O. G. Fakeyede, P. A. Okeleke, and O. R. Adaramodu, "GRC Strategies in Modern Cloud Infrastructures: A Review of Compliance Challenges," *Computer Science & IT Research Journal* 4, no. 2 (2023): 111–125.

154. K. R. Gade, "Data Governance in the Cloud: Challenges and Opportunities," *MZ Computing Journal* 4, no. 1 (2023): 1–18.

155. L. Maria Bruma, "Using Cloud Control Matrix to Evaluate Trust in Cloud Providers," in *Proceedings of the 2021 10th International Conference on Software and Computer Applications* (Association for Computing Machinery, 2021), 273–278.

156. S. K. Sahu, "Security Standards and Best Practices," in *Building Secure PHP Applications* (Apress, 2024): 249–313.

157. A. Belkum, T. T. Bachmann, G. Lüdke, et al., "Developmental Roadmap for Antimicrobial Susceptibility Testing Systems," *Nature Reviews Microbiology* 17, no. 1 (2019): 51–62.

158. Y. H. Tung, S. C. Lo, J. F. Shih, and H. F. Lin, "An Integrated Security Testing Framework for Secure Software Development Life Cycle," in *2016 18th Asia-Pacific Network Operations and Management Symposium (APNOMS)* (IEEE, 2016), 1–4.

159. M. A. Umar and C. Zhanfang, "A Study of Automated Software Testing: Automation Tools and Frameworks," *International Journal of Computer Science Engineering (IJCSE)* 6, no. 217–225 (2019): 47–48.

160. R. Akhilesh, O. Bills, N. Chilamkurti, and M. J. M. Chowdhury, "Automated Penetration Testing Framework for Smart-Home-Based IoT Devices," *Future Internet* 14, no. 10 (2022): 276.

161. V. Garousi, N. Joy, and A. B. Keleş, "AI-Powered Software Testing Tools: A Systematic Review and Empirical Assessment of Their Features and Limitations," preprint, arXiv, May 1, 2025, https://doi.org/10.48550/arXiv.2409.00411.

162. M. M. Al-Daeef, N. Basir, and M. M. Saudi, "Security Awareness Training: A Review," in *Proceedings of the World Congress on Engineering* (WCE, 2017): 5–7.

163. F. Abu-Amara, R. Almansoori, S. Alharbi, M. Alharbi, and A. Alshehhi, "A Novel SETA-Based Gamification Framework to Raise Cybersecurity Awareness," *International Journal of Information Technology* 13, no. 6 (2021): 2371–2380.

164. S. Gulati, A. Tyagi, and P. K. Goel, "Security Automation and Orchestration in the Cloud," in *Analyzing and Mitigating Security Risks in Cloud Computing* (IGI Global, 2024), 24–47.

165. J. Kinyua and L. Awuah, "AI/ML in Security Orchestration, Automation and Response: Future Research Directions," *Intelligent Automation & Soft Computing* 28, no. 2 (2021): 527–545.

166. S. M. Mohammad and L. Surya, "Security Automation in Information Technology," *International Journal of Creative Research Thoughts (IJCRT)* 6, no. 2 (2018): 901–905.

167. N. M. Yungaicela-Naula, C. Vargas-Rosales, J. A. Pérez-Díaz, and M. Zareei, "Towards Security Automation in Software Defined Networks," *Computer Communications* 183 (2022): 64–82.

168. E. Casalicchio and S. Iannucci, "The State-of-the-Art in Container Technologies: Application, Orchestration and Security," *Concurrency and Computation: Practice and Experience* 32, no. 17 (2020): e5668.

169. K. A. Torkura, M. I. Sukmana, F. Cheng, and C. Meinel, "CloudStrike: Chaos Engineering for Security and Resiliency in Cloud Infrastructure," *IEEE Access* 8 (2020): 123044–123060.

170. R. Cioffi, M. Travaglioni, G. Piscitelli, A. Petrillo, and F. De Felice, "Artificial Intelligence and Machine Learning Applications in Smart Production: Progress, Trends, and Directions," *Sustainability* 12, no. 2 (2020): 492.

171. M. Soori, B. Arezoo, and R. Dastres, "Artificial Intelligence, Machine Learning and Deep Learning in Advanced Robotics, a Review," *Cognitive Robotics* 3 (2023): 54–70.

172. M. M. Taye, "Understanding of Machine Learning With Deep Learning: Architectures, Workflow, Applications and Future Directions," *Compute* 12, no. 5 (2023): 91.

173. V. Viegas and O. Kuyucu, "Security Metrics," in *IT Security Controls: A Guide to Corporate Standards and Frameworks* (Springer, 2022), 221–244.

174. C. Kielland, *Information Security Performance Evaluation: Building a Security Metrics Library and Visualization Dashboard*" (Master's thesis (University of Oslo, 2023).

175. C. Onwubiko and A. Onwubiko, "Cyber KPI for Return on Security Investment," in *2019 International Conference on Cyber Situational Awareness, Data Analytics And Assessment (Cyber SA)* (IEEE, 2019), 1–8.

176. M. Gschwandtner, L. Demetz, M. Gander, and R. Maier, "Integrating Threat Intelligence to Enhance an Organization's Information Security Management," in *Proceedings of the 13th International Conference on Availability, Reliability and Security* (Association for Computing Machinery, 2018), 1–8.

177. S. Sultan, I. Ahmad, and T. Dimitriou, "Container Security: Issues, Challenges, and the Road Ahead," *IEEE Access* 7 (2019): 52976–52996.

178. C. Stach and B. Mitschang, "The Secure Data Container: An Approach to Harmonize Data Sharing With Information Security," in *2016 17th IEEE International Conference on Mobile Data Management (MDM)* (IEEE, 2016), 292–297.

179. B. Tak, C. Isci, S. Duri, N. Bila, S. Nadgowda, and J. Doran, "Understanding Security Implications of Using Containers in the Cloud," in *2017 USENIX Annual Technical Conference (USENIX ATC 17)* (USENIX Association, 2017), 313–319.

180. E. L. Vargas, J. Hejderup, M. Kechagia, M. Bruntink, and G. Gousios, "Enabling Real-Time Feedback in Software Engineering," in *Proceedings of the 40th International Conference on Software Engineering: New Ideas and Emerging Results* (IEEE, 2018), 21–24.

181. G. Moser, R. Vallon, M. Bernhart, and T. Grechenig, "Teaching Software Quality Assurance With Gamification and Continuous Feedback Techniques," in *2021 IEEE Global Engineering Education Conference (EDUCON)* (IEEE, 2021), 505–509.

182. A. Dawoud, S. Finster, N. Coppik, and V. Ashiwal, "Better Left Shift Security! Framework for Secure Software Development," in *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)* (IEEE, 2024), 642–649.

183. D. Gonzalez, P. P. Perez, and M. Mirakhorli, "Barriers to Shift-Left Security: The Unique Pain Points of Writing Automated Tests Involving Security Controls," in *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)* (Association for Computing Machinery, 2021), 1–12.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

184. M. A. Storey, N. A. Ernst, C. Williams, and E. Kalliamvakou, "The Who, What, How of Software Engineering Research: A Socio-Technical Framework," *Empirical Software Engineering* 25 (2020): 4097–4129.

185. B. Turnheim and B. K. Sovacool, "Exploring the Role of Failure in Socio-Technical Transitions Research," *Environmental Innovation and Societal Transitions* 37 (2020): 267–289.

186. M. Sánchez-Gordón and R. Colomo-Palacios, "Characterizing DevOps Culture: A Systematic Literature Review," in *Software Process Improvement and Capability Determination: 18th International Conference, SPICE 2018* (Springer, 2018), 3–15.

187. S. Kanungo, "Security Challenges and Solutions in Multi-Cloud Environments," *Stochastic Modelling and Computational Sciences* 3, no. 2 (2023): 139–146.

188. T. E. Lander, Jr., "Multi-Cloud Architecture Attacks Through Application Programming Interfaces" (Graduate thesis, James Worth Bagley College of Engineering, 2024).

189. B. Desai and K. Patil, "Demystifying the Complexity of Multi-Cloud Networking," *Asian American Research Letters Journal* 1, no. 4 (2024): 2–6.

190. N. Shinde and P. Kulkarni, "Cyber Incident Response and Planning: A Flexible Approach," *Computer Fraud & Security* 2021, no. 1 (2021): 14–19.

191. N. A. Z. Farok and M. F. Zolkipli, "Incident Response Planning and Procedures," *Borneo International Journal* 7, no. 2 (2024): 69–76.

192. M. Ricks, "Building Capability and Community Through Cyber-Incident Response Exercises," *Journal of Business Continuity & Emergency Planning* 18, no. 1 (2024): 49–58.

193. X. Zhou, Y. Jin, H. Zhang, S. Li, and X. Huang, "A Map of Threats to Validity of Systematic Literature Reviews in Software Engineering," in *2016 23rd Asia-Pacific Software Engineering Conference (APSEC)* (IEEE, 2016), 153–160.

194. M. Unterkalmsteiner, T. Gorschek, A. M. Islam, C. K. Cheng, R. B. Permadi, and R. Feldt, "Evaluation and Measurement of Software Process Improvement—A Systematic Literature Review," *IEEE Transactions on Software Engineering* 38, no. 2 (2011): 398–424.

20477481, 2025, 6, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70029, Wiley Online Library on [19/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License